

Учреждение образования «БЕЛОРУССКИЙ
ГОСУДАРСТВЕННЫЙ ТЕХНОЛОГИЧЕСКИЙ УНИВЕРСИТЕТ»

Факультет Информационных технологий
Кафедра Информационных систем и технологий
Специальность 1-40 05 01 Информационные системы и технологии
Специализация 1-40 05 01-03 Информационные системы и технологии (изда-
тельско-полиграфический комплекс)

ПОЯСНИТЕЛЬНАЯ ЗАПИСКА ДИПЛОМНОГО ПРОЕКТА НА ТЕМУ:

Веб-приложение для краудсорсинга идей «IdeaFlow»

Обучающийся _____ 4 курс, 2 группа _____ А.Д. Телица
курс, группа _____ дата _____ подпись _____ И. О. Ф.

Руководитель
дипломного проекта _____ ст. преп. _____ Д.В. Сазонова
должность, ученая степень, ученое звание _____ дата _____ подпись _____ И. О. Ф.

Заведующий
кафедрой _____ К.Т.Н. _____ Е.А. Блинова
должность, ученая степень, ученое звание дата подпись И. О. Ф.

Консультант _____ ст. преп. _____ А.С. Соболевский
должность, ученая степень, ученое звание _____ дата _____ подпись И. О. Ф.

Нормоконтролер _____ асс. _____ А.Н. Николайчук
должность, ученая степень, ученое звание _____ дата _____ подпись И. О. Ф.

Дипломный проект защищен с оценкой

Председатель ГЭК _____ к.т.н., доцент _____ В.К. Дюбков
должность, ученая степень, ученое звание _____ дата _____ подпись _____ И. О. Ф.

Минск 2026

Министерство образования Республики Беларусь
 Учреждение образования «БЕЛОРУССКИЙ
 ГОСУДАРСТВЕННЫЙ ТЕХНОЛОГИЧЕСКИЙ УНИВЕРСИТЕТ»

Кафедра Информационных систем и технологий

УТВЕРЖДАЮ
 Заведующий кафедрой ИСИТ
 _____ Е.А. Блинова
 « ____ » _____ 2026 г.

**ЗАДАНИЕ
 НА ДИПЛОМНЫЙ ПРОЕКТ**

обучающемуся Телица Анне Дмитриевне
 (фамилия, имя, отчество)

Курс 4 Группа 2
 Специальность 1-40 05 01 Информационные системы и технологии
 Специализация 1-40 05 01-03 Информационные системы и технологии (изда-
 тельско-полиграфический комплекс)
 Тема дипломного проекта: Веб-приложение для краудсорсинга идей
 «IdeaFlow»

утверждена ректором БГТУ 5 февраля 2026 г. № 15-С

Исходные данные к дипломному проекту:

- поддержка трех ролей пользователей: Гость, Пользователь, Администратор;
- функционал Гостя: регистрация с подтверждением email, авторизация;
- функционал Пользователя: взаимодействие с темами и идеями, избранное, лайки, комментарии, система достижений, личная статистика;
- функционал Администратора: полное управление контентом, модерация тем, блокировка пользователей, техподдержка, аналитика;
- модуль анализа со статистикой активности;
- система достижений пользователя;
- раздел «Популярные темы сообщества» на главной странице;
- личная статистика для пользователей.

Программная платформа и технологии: Docker, NestJS 11.1.9, TypeScript 5.9.3, TypeORM 0.3.27, PostgreSQL 15, React.js 18.2.0, React Router DOM 6.20.0, JWT (Passport JWT 4.0.1), Nodemailer 6.9.7, Axios 1.6.0, bcrypt 6.0.0, Yup 1.3.0, React Hook Form 7.49.2.

Краткое содержание расчетно-пояснительной записки:

1) оглавление;

Оглавление

Реферат	5
Abstract	6
Введение	7
1 Постановка задачи и обзор аналогичных решений	8
1.1 Аналитический обзор аналогов	8
1.1.1 Пряники	9
1.1.2 IdeaScale	9
1.1.3 Платформа Brightidea	10
1.2 Патентный поиск	11
1.3 Выводы по разделу	13
2 Проектирование веб-приложения.....	14
2.1 Выбор средств реализации	14
2.2 Архитектурная схема приложения	15
2.3 Функциональные требования и варианты использования	15
2.4 Диаграмма развертывания	17
2.5 Проектирование базы данных	19
2.6 Контейнеризация.....	23
2.7 Проектирование основных алгоритмов	24
2.7 Выводы по разделу	24
3 Разработка веб-приложения.....	25
3.1 Разработка серверной части.....	25
3.1.1 Реализация модуля авторизации.....	28
3.1.2 Реализация модуля управления темами для обсуждения	30
3.1.3 Реализация механизма блокировки пользователей.....	31
3.1.4 Реализация подтверждения email и восстановления пароля	32
3.1.5 Реализация механизма реакций	33
3.1.6 Реализация системы вложенных комментариев.....	34
3.1.7 Система достижений и бейджей.....	35
3.2. Разработка клиентской части	37
3.2.1 Организация модулей представления.....	38
3.2.2 Организация асинхронного взаимодействия с сервером	39
3.2.3 Разработка основного компонента административной панели.....	41
3.2.4 Библиотека переиспользуемых UI-компонентов	42
3.2.5 Управление стилями	42
3.2.6 Корневые модули и конфигурация	42
3.3. Контейнеризация приложения с использованием Docker.....	43
3.4 Выводы по разделу	44
4 Тестирование веб-приложения	45

					ДП 00.00.ПЗ									
		ФИО	Подпись	Дата										
Разраб.		Телица А.Д.			Оглавление				Лит.		Лист		Листов	
Пров.		Сазонова Д.В.							у		1		3	
Н. контр.		Николайчук А.Н.							БГТУ 1-40 05 01, 2026					
Утв.		Блинова Е.А.												

4.1 Тестирование модуля регистрации и авторизации	45
4.2 Тестирование модуля тем для обсуждения	47
4.3 Тестирование модуля идей к темам для обсуждения	47
4.3.1 Валидация контента при публикации идеи	47
4.3.2 Система реакций и геймификации	48
4.3.3 Личный кабинет пользователя	49
4.4 Тестирование панели администратора	50
4.4.1 Управление пользовательской базой	50
4.4.2 Управление темами для обсуждения	51
4.4.3 Модерация идей и комментариев	52
4.4.4 Просмотр статистики платформы	53
4.5 Выводы по разделу	53
5 Руководство пользователя	54
5.1 Руководство пользователя для роли «Пользователь»	54
5.2 Руководство пользователя для «Администратора»	58
5.3 Выводы по разделу	62
6 Техничко-экономическое обоснование проекта	63
6.1 Общая характеристика разрабатываемого продукта	63
6.2 Исходные данные для проведения расчетов	63
6.3 Методика обоснования цены	64
6.4 Расчет затрат рабочего времени на разработку программного средства	65
6.5 Расчет основной заработной платы	65
6.6 Расчет дополнительной заработной платы	67
6.7 Расчет отчислений на социальные цели	67
6.8 Расчет расходов на материалы	68
6.9 Расчет расходов на специальное оборудование и платные услуги	68
6.10 Расчет прочих прямых затрат	68
6.11 Расчет общепроизводственных и общехозяйственных расходов	68
6.12 Расчет суммы расходов на разработку программного средства	69
6.13 Расчет расходов на сопровождение и адаптацию	69
6.14 Расчет полной себестоимости программного средства	70
6.15 Определение цены и оценка эффективности	70
6.16 Вывод по разделу	73
Заключение	75
Список использованных источников	76
ПРИЛОЖЕНИЕ А Архитектурная схема приложения	78
ПРИЛОЖЕНИЕ Б Диаграмма вариантов использования	79
ПРИЛОЖЕНИЕ В Диаграмма развертывания	80
ПРИЛОЖЕНИЕ Г Логическая схема базы данных	81
ПРИЛОЖЕНИЕ Д Блок-схема алгоритма создания новой темы	82
ПРИЛОЖЕНИЕ Е Экранная форма главной страницы	83
ПРИЛОЖЕНИЕ Ж Таблица расчета затрат на разработку	84
ПРИЛОЖЕНИЕ И Реализация сидера учетной записи администратора	85
ПРИЛОЖЕНИЕ К Файл оркестрации контейнеров	86

Реферат

Пояснительная записка дипломного проекта содержит 77 страницы пояснительной записки, 18 таблиц, 15 формул, 39 иллюстраций, 33 источников литературы, 9 приложений.

ВЕБ-ПРИЛОЖЕНИЕ, КРАУДСОРСИНГ, ИДЕИ, ТЕМЫ, NESTJS, REACT, TYPESCRIPT, POSTGRESQL, DOCKER

Целью дипломного проекта является разработка веб-приложения для краудсорсинга идей «IdeaFlow», предназначенного для организации, структурирования и обсуждения пользовательских идей, а также обеспечивающего модерацию контента, анализ активности пользователей и разграничение прав доступа.

Первая глава содержит анализ предметной области и обзор существующих аналогов в сегменте краудсорсинговых платформ.

Вторая глава описывает проектирование веб-приложения, включая архитектуру системы, логическую модель базы данных, схему развертывания приложения, блок-схему алгоритма, диаграмму вариантов использования и обоснование выбранных технологий.

Третья глава посвящена процессу реализации приложения: детально рассматриваются разработанные модули – аутентификация, управление темами и идеями, система комментариев, блок достижений и административная панель.

В четвертой главе представлены результаты тестирования веб-приложения и анализ его поведения при различных сценариях использования.

В пятая глава содержит руководство пользователя с описанием функциональных возможностей системы и порядка взаимодействия с интерфейсом.

Шестая глава включает технико-экономическое обоснование разработки программного продукта.

Объем графической части дипломного проекта составляет 1,25 листа формата A1.

					ДП 00.00.ПЗ		
		ФИО	Подпись	Дата	Реферат		
Разраб.	Телица А.Д.						
Пров.	Сазонова Д.В.						
Н. контр.	Николайчук А.Н.						
Утв.	Блинова Е.А.				БГТУ 1-40 05 01, 2026		
					Лит.	Лист	Листов
					у	1	1

Abstract

The thesis project's explanatory note consists of 77 pages of explanatory text, 18 tables, 15 formulas, 39 illustrations, 33 literature sources, and 9 appendices.

WEB APPLICATION, CROWDSOURCING, IDEAS, TOPICS, NESTJS, REACT, TYPESCRIPT, POSTGRES SQL, DOCKER

The aim of the diploma project is to develop the «IdeaFlow» веб application for idea crowdsourcing, intended for organizing, structuring, and discussing user ideas, as well as providing content moderation, user activity analysis, and access rights management.

The first chapter provides an analysis of the subject area and an overview of existing counterparts in the crowdsourcing platform segment.

The second chapter describes the design of the web application, including the system architecture, the logical database model, the application deployment diagram, the algorithm flowchart, the use case diagram, and the rationale for the chosen technologies.

The third chapter is devoted to the application implementation process: it examines in detail the developed modules – authentication, topic and idea management, the comment system, the achievements section, and the admin panel.

The fourth chapter presents the results of testing the веб application and an analysis of its behavior under various usage scenarios.

The fifth chapter contains a user guide describing the system's functional capabilities and how to interact with the interface.

The sixth chapter includes a technical and economic justification for the development of the software product.

The graphic portion of the thesis project comprises 1.25 A1-size sheets.

					<i>ДП 00.00.ПЗ</i>		
		ФИО	Подпись	Дата			
Разраб.		<i>Телица А.Д.</i>			<i>Abstract</i>		
Пров.		<i>Сазонова Д.В.</i>					
Н. контр.		<i>Николайчук А.Н.</i>					
Утв.		<i>Блинова Е.А.</i>					
						Лит.	Лист
						у	1
						Листов	
						1	
						<i>БГТУ 1-40 05 01, 2026</i>	

Введение

Актуальность данного проекта заключается в создании универсальной и безопасной веб-платформы – «IdeaFlow», которая систематизирует процесс краудсорсинга, обеспечивает контроль качества контента и вовлекает пользователей через удобный и функциональный интерфейс. В современном цифровом мире, где скорость генерации инноваций и коллективный интеллект становятся ключевыми конкурентными преимуществами, эффективные инструменты для сбора и структурирования идей критически важны для бизнеса, образовательных учреждений и различных общественных инициатив.

Целью дипломного проектирования является разработка веб-приложения для краудсорсинга «IdeaFlow», предназначенного для публикации, обсуждения и оценки идей пользователей с разграничением прав доступа на основе ролей (гость, пользователь, администратор).

Достижение указанной цели предполагает решение таких задач:

- выполнить постановку задачи;
- провести аналитический обзор аналогичных решений;
- спроектировать архитектуру веб-приложения, структуру базы данных и основные сценарии взаимодействия пользователей с системой;
- разработать веб-приложение «IdeaFlow» с реализацией серверной части, клиентского интерфейса, механизмов аутентификации, модерации, аналитики, геймификации и техподдержки;
- провести тестирование веб-приложения и проверить корректность работы основных функциональных возможностей системы;
- разработать руководство пользователя, описывающее порядок эксплуатации веб-приложения и работу с его основными функциями;
- выполнить технико-экономическое обоснование проекта, включая оценку трудоемкости, затрат на разработку и экономической целесообразности создания веб-приложения.

Результатом выполнения дипломного проекта является разработанное веб-приложение, реализующее функционал платформы для управления идеями, а также демонстрирующее практическое применение современных веб-технологий при создании распределенных информационных систем.

					ДП 00.00.ПЗ		
		ФИО	Подпись	Дата			
Разраб.	Телица А.Д.				Введение		
Пров.	Сазонова Д.В.						
Н. контр.	Николайчук А.Н.						
Утв.	Блинова Е.А.						
					Лит.	Лист	Листов
					у	1	1
					БГТУ 1-40 05 01, 2026		

1 Постановка задачи и обзор аналогичных решений

Важным этапом разработки любого нового продукта является анализ существующих на рынке аналогов. Он позволяет определить, какие решения уже представлены потребителям, выявить их сильные и слабые стороны, а также определиться с направлениями для усовершенствований.

Краудсорсинговая платформа – это веб-приложение, которое позволяет организовать коллективную работу над идеями. Такие системы особенно востребованы в корпоративной среде для инноваций, в образовательных учреждениях для организации проектной деятельности, а также в государственном управлении для сбора предложений активных граждан.

На текущий момент рынок краудсорсинговых платформ представляет собой не так много решений с высоким уровнем функциональности. Многие из них не могут предложить полноценного комплекса инструментов, сочетающего в себе удобство, гибкие настройки модерации и подробную аналитику. Это мотивирует создавать новые и качественные продукты в этой сфере.

Задача проекта: разработать архитектуру приложения, взаимодействие с которой будет понятно любому пользователю. Построить базу данных и выполнить тестирование готового продукта.

Должны быть выполнены следующие требования:

- обеспечивать возможность регистрации и авторизации;
- поддерживать роли администратора, пользователя и гостя;
- управление темами для обсуждения;
- модерацию контента для администратора;
- возможность предложения идей;
- возможность взаимодействия с идеями для пользователей;
- предоставление статистики и аналитики для пользователей;
- настройки приватности обсуждений для администраторов;
- блокировка/разблокировка пользователей администратором.

1.1 Аналитический обзор аналогов

На основе изучения существующих краудсорсинговых платформ и анализа поставленных задач были определены ключевые требования к разрабатываемой системе. Изучение существующих решений позволило выделить ключевые функциональные требования к системе. Оценка аналогичных платформ позволила сформировать перечень требований к разрабатываемой системе.

					ДП 01.00.ПЗ		
		ФИО	Подпись	Дата	1 Постановка задачи и обзор аналогичных решений		
Разраб.		Телица А.Д.					
Пров.		Сазонова Д.В.					
Н. контр.		Николайчук А.Н.					
Утв.		Блинова Е.А.					
					Лит.	Лист	Листов
					у	1	4
					БГТУ 1-40 05 01, 2026		

1.1.1 Пряники

В качестве актуального аналога рассматривается краудсорсинговая платформа «Пряники» [6], позиционирующая себя как инструмент для сбора и развития идей. Платформа предлагает лаконичный интерфейс с системой карточек для предложения идей, базовыми возможностями голосования и комментирования. Основной упор делается на визуальную простоту и минималистичный дизайн, что снижает порог входа для новых пользователей.

К сильным сторонам решения можно отнести интуитивно понятный процесс добавления идей, систему реакций для быстрой обратной связи и адаптивный интерфейс. Однако функционал платформы ограничен: наблюдается недостаток инструментов для структурирования контента – нет категоризации тем, тегов или удобных тематических фильтров.

Интерфейс платформы представлен на рисунке 1.1.

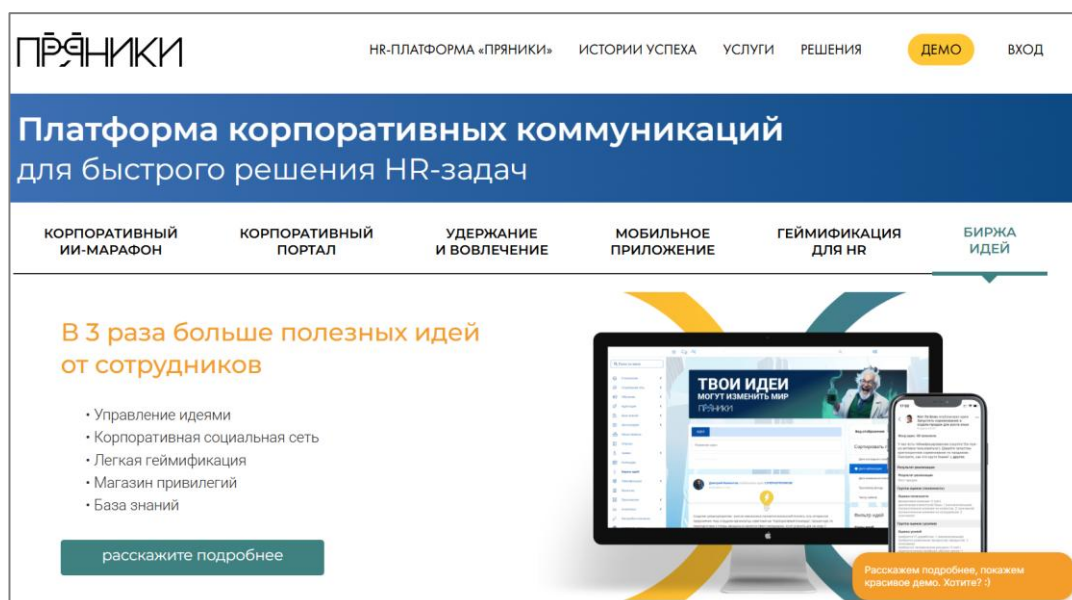


Рисунок 1.1 – Платформа «Пряники»

Из опыта данной платформы для проекта «IdeaFlow» было принято решение заимствовать принцип минималистичного представления карточек с идеями и систему реакций для быстрой пользовательской обратной связи.

1.1.2 IdeaScale

IdeaScale [7] является одной из ведущих краудсорсинговых платформ на мировом рынке, предоставляющей комплексные решения для управления инновациями в корпоративном и государственном секторах.

Главный экран IdeaScale устроен логично. В центре внимания – текущие проекты по сбору идей, показатели активности и инструменты для модерации. В верхней части экрана вынесены основные разделы: аналитика, настройки проектов и управление пользователями.

Данный подход делает работу с платформой удобной даже при большом количестве идей. Интерфейс не перегружен лишними элементами, что особенно важно при работе с постоянным потоком новых предложений.

Интерфейс платформы представлен на рисунке 1.2

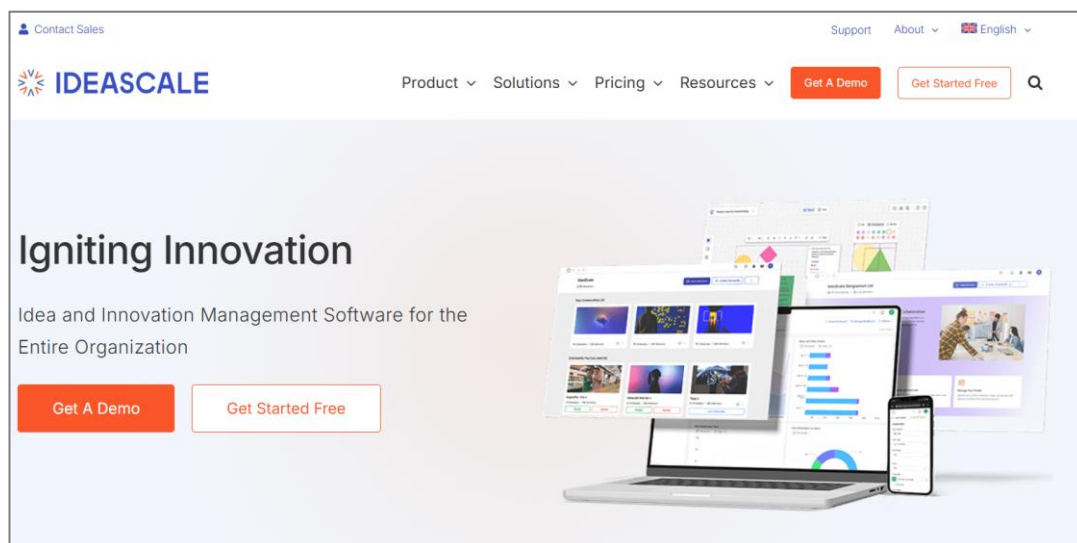


Рисунок 1.2 – Платформа IdeaScale

В рамках проекта «IdeaFlow» целесообразно применение подхода к реализации ролевой модели и базовой аналитики, а также учет эргономичности интерфейса и доступности пользовательского взаимодействия.

1.1.3 Платформа Brightidea

Brightidea [8] представляет собой корпоративное решение для управления инновационными идеями на основе краудсорсинга. Сервис ориентирован на крупные компании и предоставляет комплекс инструментов для сбора, систематизации и последующей обработки пользовательских инициатив.

Brightidea позволяет централизованно отслеживать поступающие предложения и контролировать этапы их рассмотрения. Решение рассчитано на взаимодействие большого количества участников, поэтому поддерживает распределение материалов по направлениям и инициативам. Такой подход упрощает поиск нужной информации и помогает организовать работу в рамках конкретных задач. За счет встроенных инструментов экспертного анализа администраторы и специалисты могут быстрее определять наиболее перспективные предложения. Таким образом, сервис охватывает полный цикл работы – от публикации инициативы до анализа полученных результатов.

Пользователи могут размещать предложения по соответствующим темам, принимать участие в обсуждениях и совместно дорабатывать опубликованные материалы. Одной из ключевых особенностей является гибкая настройка процессов модерации и экспертного рассмотрения.

Интерфейс сочетает корпоративный дизайн с интуитивной навигацией. Основной экран предоставляет доступ к тематическим разделам, аналитике и

инструментам управления. Каждая тема содержит информацию о текущем статусе, экспертных оценках и комментариях участников.

Для администраторов предусмотрены расширенные аналитические возможности, включая дашборды и систему отчетности. Также поддерживаются мобильный доступ и интеграция с корпоративными системами через API.

К недостаткам можно отнести перегруженность интерфейса и избыточное количество визуальных элементов. Большой объем информации на экране затрудняет восприятие и навигацию. Особенно это заметно на главной странице, где яркие изображения отвлекают внимание от текстового содержимого.

Внедрение Brightidea характеризуется значительной трудоемкостью, обусловленной обширным функционалом и многообразием конфигурационных параметров. Тем не менее, по мере освоения инструментария сервис способствует повышению эффективности управления инновационными процессами и активизации инициативной деятельности внутри организации.

Интерфейс платформы представлен на рисунке 1.3

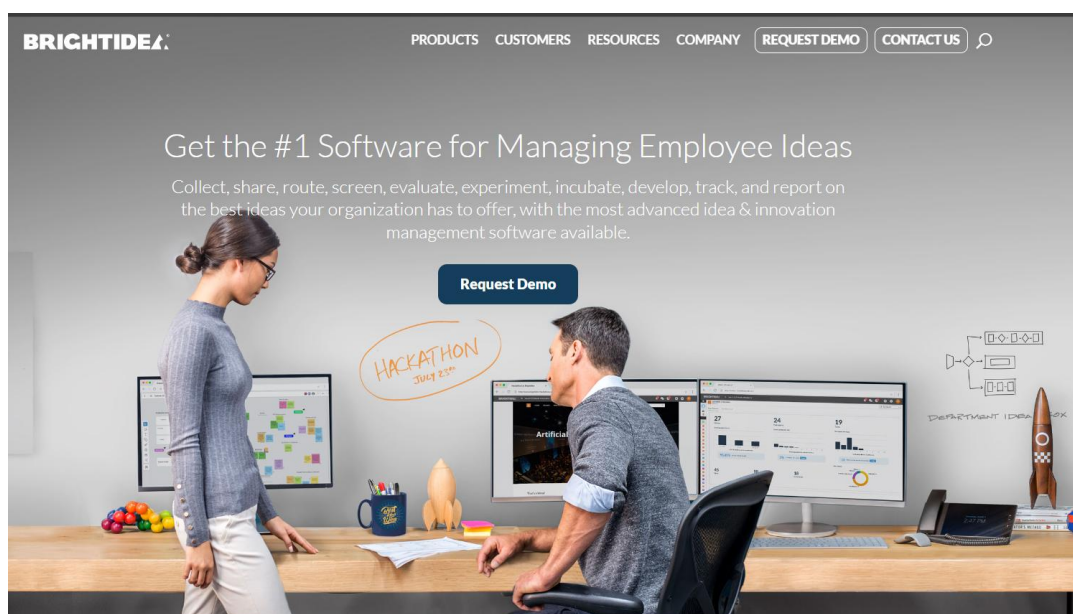


Рисунок 1.3 – Платформа Brightidea

Для проекта IdeaFlow целесообразно адаптировать модульную архитектуру, обеспечив при этом упрощенный интерфейс и улучшенную навигацию. Сохраняя возможность тематической организации идей и расширенной аналитики, важно оптимизировать визуальное представление данных и сократить количество элементов управления. Особое внимание будет уделено упрощению процессов модерации и совместной работы над идеями.

1.2 Патентный поиск

В данном подразделе представлены результаты патентного исследования, выполненного в рамках разработки веб-приложения для краудсорсинга

идей «IdeaFlow». Поиск проводился с использованием базы данных Национального центра интеллектуальной собственности Республики Беларусь. В ходе исследования были рассмотрены патентные документы, связанные с механизмами голосования, экспертной оценки и обработки пользовательских решений.

В результате поиска прямых патентных аналогов веб-приложения для краудсорсинга идей с поддержкой обсуждений, модерации, аналитики активности, геймификации и разграничения прав доступа в Республике Беларусь выявлено не было. При этом были найдены отдельные технические решения, близкие к некоторым функциональным аспектам разрабатываемой системы. Сведения о найденных документах представлены в таблицах 1.1 и 1.2.

Первым найденным решением является патентный документ, посвященный способу голосования для осуществления избирательной процедуры; сведения о нем представлены в таблице 1.1.

Таблица 1.1 – Описание патента №1

Наименование	Номер патента	Опубликовано	Авторы
«Способ голосования для осуществления избирательной процедуры»	а 20200024	30.08.2021	Буян Алексей Петрович

В данном патентном документе рассматривается способ голосования для реализации избирательной процедуры, связанный с обработкой и фиксацией результатов выбора. Для проекта «IdeaFlow» он представляет интерес только с точки зрения общего механизма учета пользовательского выбора, поскольку в приложении элементы коллективной оценки используются не для выборов, а для определения интереса, популярности и значимости идей внутри платформы.

Вторым найденным решением является патентный документ, связанный с экспертной оценкой объектов; сведения о нем представлены в таблице 1.2.

Таблица 1.2 – Описание патента №2

Наименование	Номер патента	Опубликовано	Авторы
«Способ экспертной оценки объектов»	а 20120352	12.03.2012	Егоренков Николай Иванович; Казаков Андрей Сергеевич.

В данном патентном документе рассматривается способ экспертной оценки объектов, основанный на их анализе и сравнении по заданным критериям. Для проекта «IdeaFlow» он представляет интерес в части оценки предложений, формирования рейтингов и анализа пользовательской активности. Однако разрабатываемое приложение шире по функциональности, так как

объединяет публикацию идей, обсуждение, модерацию, ролевую модель, статистику, достижения и поддержку пользователей.

Таким образом, проведенный патентный поиск показал, что в базе данных Национального центра интеллектуальной собственности Республики Беларусь не выявлено прямых аналогов веб-приложения «IdeaFlow». Найденные патентные документы отражают лишь отдельные функциональные направления, такие как голосование и экспертная оценка объектов. Разрабатываемое приложение отличается более широким функциональным охватом, поскольку объединяет сбор, структурирование, обсуждение и анализ пользовательских идей в рамках единой цифровой платформы с поддержкой модерации, геймификации и разграничения прав доступа.

1.3 Выводы по разделу

В данном разделе были рассмотрены существующие краудсорсинговые платформы, такие как «Пряники», IdeaScale и Brightidea. Исходя из анализа этих аналогов, можно выделить несколько идей для разработки «IdeaFlow». Например, платформа должна предоставлять трехуровневую модель доступа, как в IdeaScale, а также обеспечивать удобное управление контентом и гибкую настройку тем и сроков, аналогично другим современным системам.

Также в данном разделе рассмотрен основной функционал, который должен присутствовать в конечном приложении. Этот функционал рассчитан на то, чтобы пользователи могли легко взаимодействовать с платформой, создавать и обсуждать идеи, загружать медиафайлы и получать статистику по активности. Была приведена диаграмма вариантов использования, на которой наглядно показано распределение функций между ролями пользователей и основные возможности системы.

Также в рамках раздела был проведен патентный поиск, в ходе которого рассмотрены решения, связанные с краудсорсингом, коллективной оценкой и ранжированием идей. Анализ показал, что «IdeaFlow» объединяет данные возможности в единую систему с поддержкой модерации, аналитики, геймификации и разграничения прав доступа.

2 Проектирование веб-приложения

2.1 Выбор средств реализации

Выбор технологического стека был обусловлен требованиями к производительности, масштабируемости, безопасности и удобству разработки. Платформа «IdeaFlow» реализована на основе современных инструментов, объединенных в единую контейнеризированную среду.

Клиентская часть реализована на React.js, что обеспечивает быстрый и отзывчивый асинхронный интерфейс. Для навигации между страницами используется React Router DOM [9], а для выполнения HTTP-запросов к серверу – библиотека Axios [10]. Управление формами и их валидация реализованы с помощью React Hook Form [11] и Yup [12]. Пользователю доступны функции добавления идей, комментариев, реакций, управления избранными темами и просмотра статистики. Взаимодействие с сервером осуществляется по защищенному протоколу HTTPS [13].

Серверная часть построена на фреймворке Nest.js с использованием TypeScript, что обеспечивает строгую типизацию и модульную архитектуру. Для работы с данными выбран TypeORM [14] – ORM, поддерживающий миграции и удобную работу с сущностями. Валидация данных на сервере осуществляется с помощью библиотек class-validator [15] и class-transformer [16]. Безопасность паролей обеспечивается хешированием с использованием библиотеки bcrypt [17]. Для отправки email-уведомлений, включая подтверждение регистрации и восстановление пароля, применяется Nodemailer [18]. Авторизация реализована на основе JWT с использованием Passport JWT [19]. Серверная часть, как и остальные компоненты, контейнеризирована с помощью Docker, что гарантирует стабильность и воспроизводимость окружения на любом этапе развертывания. Сервер реализует REST API и всю бизнес-логику приложения: авторизацию, модерацию контента, управление ролями, систему достижений, аналитику и формирование статистики. Взаимодействие с базой данных осуществляется по стандартному протоколу TCP.

В качестве системы управления базами данных выбрана PostgreSQL [20]. Она также развернута в отдельном контейнере и обеспечивает надежное хранение данных, поддержку сложных запросов, встроенных типов данных, таких как UUID для уникальных идентификаторов и JSON для гибкого хранения структур. PostgreSQL отвечает требованиям ACID, что критически важно для обеспечения целостности данных в многопользовательской системе. Такой набор технологий обеспечивает разделение клиентской, серверной и инфраструктурной частей, упрощая сопровождение, развитие и масштабирование проекта.

					ДП 02.00.ПЗ		
		ФИО	Подпись	Дата	2 Проектирование веб-приложения		
Разраб.	Телица А.Д.						
Пров.	Сазонова Д.В.						
Н. контр.	Николайчук А.Н.						
Утв.	Блинова Е.А.						
					Лит.	Лист	Листов
					у	1	8
					БГТУ 1-40 05 01, 2026		

2.2 Архитектурная схема приложения

Архитектурная схема приложения [21] представляет собой визуальное описание структуры системы и размещения ее программных компонентов. Она отражает распределение элементов приложения между клиентской частью, сервером и базой данных, а также показывает основные каналы их взаимодействия. Такая схема упрощает понимание внутренней организации системы, выявление связей между компонентами и планирование последующего развертывания приложения. Часть схемы архитектуры разработанного веб-приложения представлена на рисунке 2.1, полная схема представлена в приложении А.

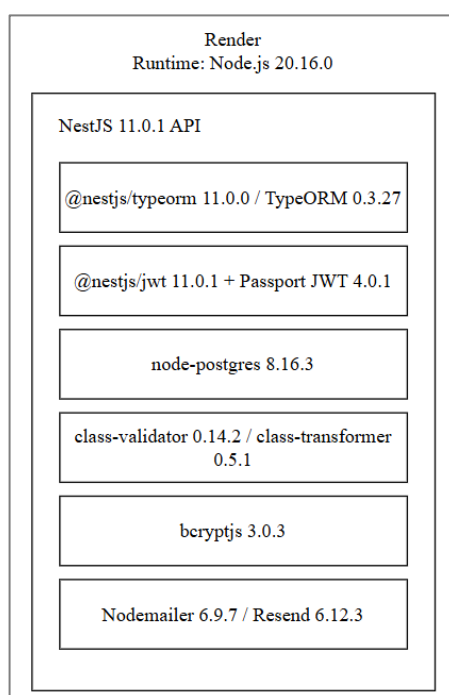


Рисунок 2.1 – Архитектурная схема приложения

Архитектурная схема показывает, что веб-приложение «IdeaFlow» построено на трехзвенной архитектуре с разделением клиентского уровня, серверной части и базы данных. Такое решение обеспечивает распределение ответственности между компонентами, контейнеризированное развертывание и защищенный обмен данными.

2.3 Функциональные требования и варианты использования

На основе анализа предметной области сформулированы функциональные требования к веб-приложению «IdeaFlow». Основной задачей системы является создание управляемой среды для коллективной генерации идей. В архитектуре предусмотрена трехуровневая модель разграничения прав доступа: администратор, зарегистрированный пользователь и гость. Для каждой роли определены допустимые действия и ограничения, что обеспечивает безопасность и соблюдение принципа минимальных привилегий.

Администратор получает полный контроль над контентом и пользователями системы. Он может создавать, редактировать, удалять и модерировать темы, управлять идеями, комментариями и учетными записями, включая блокировку пользователей, обрабатывать обращения через систему поддержки, а также использовать модуль анализа активности для просмотра статистики и оценки вовлеченности пользователей.

Для зарегистрированного пользователя предусмотрены просмотр одобренных публичных тем, работа с избранным, публикация, редактирование и удаление собственных идей, закрепление идей, оценка материалов других участников с помощью реакций, комментирование с поддержкой иерархических обсуждений, просмотр статистики личной активности и предложение новых тем с последующей модерацией. Такие возможности обеспечивают полноценное участие пользователя в обсуждении и развитии идей.

Гость может войти в систему с использованием электронной почты и пароля либо зарегистрировать учетную запись с подтверждением адреса электронной почты через отправленный код.

Для формализации расширенных административных функций разработана диаграмма вариантов использования в нотации UML. Диаграмма на рисунке 2.2 отражает только функции администратора, недоступные зарегистрированным пользователям и гостям. Полная диаграмма вариантов использования со всеми ролями системы представлена в приложении Б.

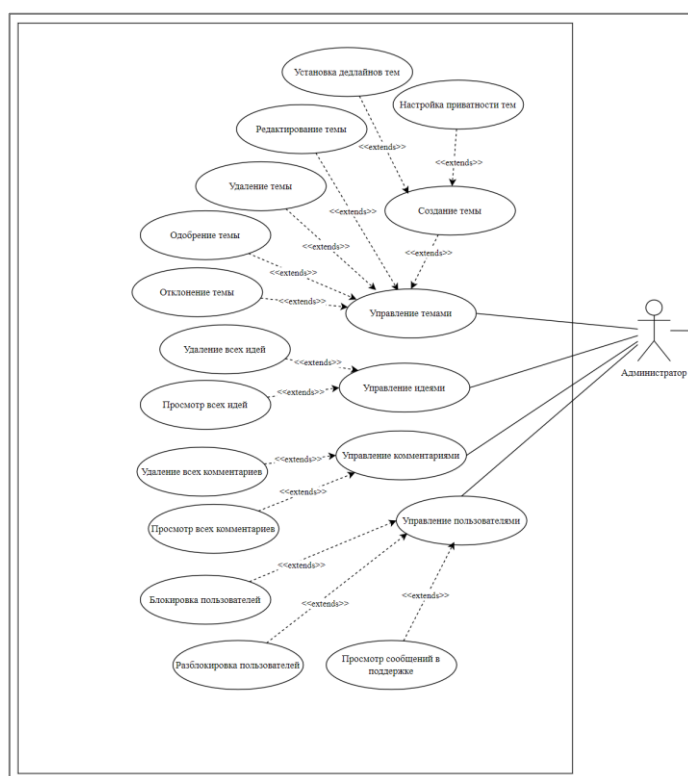


Рисунок 2.2 – Диаграмма вариантов использования расширенных функций

Диаграмма UML [22] вариантов использования позволяет идентифицировать доступные действия для каждой роли в веб-приложении.

2.4 Диаграмма развертывания

Диаграмма развертывания является диаграммой UML, отражающей физическое размещение программных компонентов системы на аппаратных и программных узлах, а также каналы взаимодействия между ними. В отличие от архитектурной схемы, описывающей логическую структуру приложения, диаграмма развертывания показывает, в каком окружении выполняются основные компоненты системы и каким образом они связаны между собой. Часть диаграммы развертывания веб-приложения «IdeaFlow» представлена на рисунке 2.3, полная диаграмма приведена в приложении В.

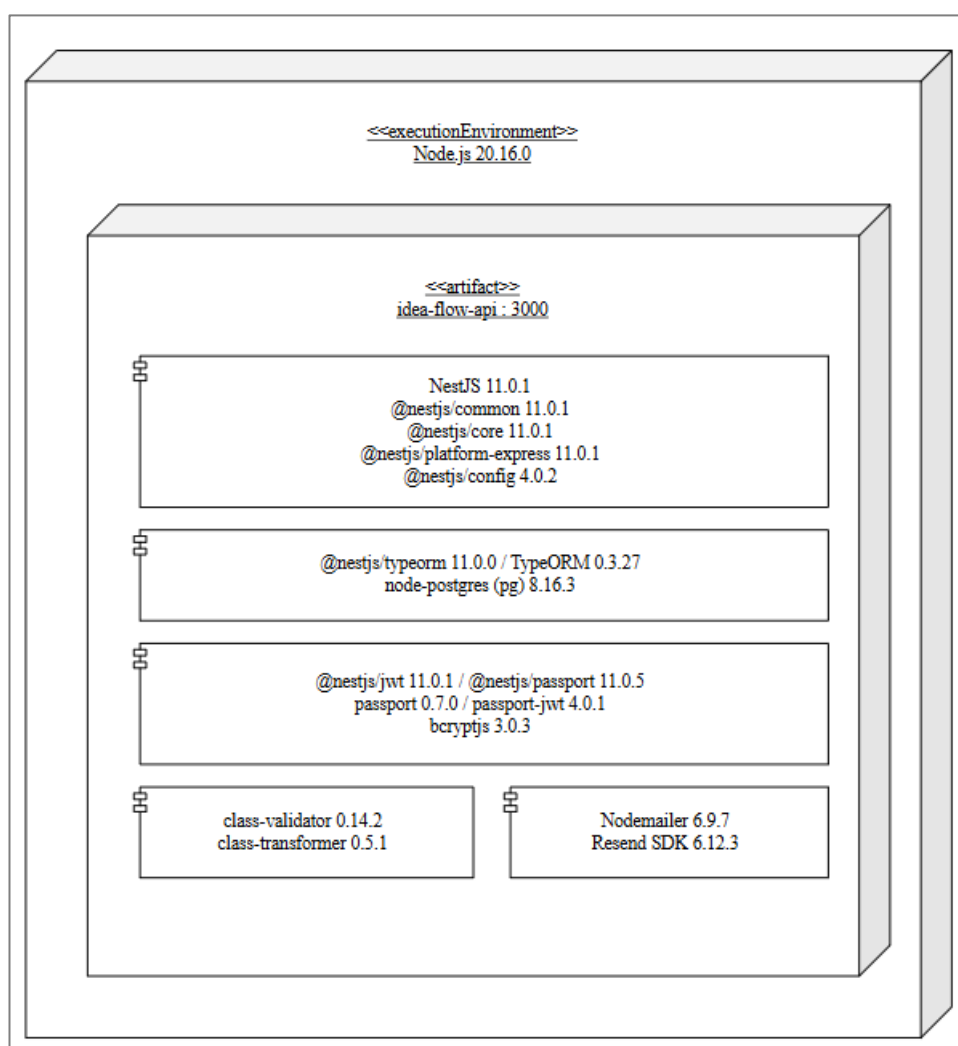


Рисунок 2.3 – Серверный блок диаграммы развертывания

Диаграмма включает несколько основных узлов: пользовательское устройство, устройство администратора, внешний почтовый сервис, серверный хост и сервер базы данных. На пользовательском устройстве запускается браузер Microsoft Edge версии 134.0.3124.51, через который пользователь получает доступ к клиентской части приложения. В браузере выполняется React-приложение, построенное с использованием React.js, React Router DOM, Axios, React Hook Form, Yup и других клиентских библиотек.

Устройство администратора также взаимодействует с системой через браузер и использует административную часть веб-интерфейса. Через нее выполняется управление пользователями, темами, идеями, комментариями, обращениями в техническую поддержку и просмотр аналитических данных. Взаимодействие пользовательского и административного клиентских устройств с серверной частью осуществляется по защищенному протоколу HTTPS.

На серверном хосте развернута среда контейнеризации Docker Compose, в которой размещены основные программные компоненты системы. Контейнер клиентской части содержит собранное React-приложение и веб-сервер Nginx, обеспечивающий выдачу статических файлов интерфейса. Контейнер серверной части содержит backend-приложение, реализованное на Nest.js и TypeScript. В его состав входят модули авторизации, управления пользователями, темами, идеями, комментариями, реакциями, достижениями, поддержкой и аналитикой активности.

Серверное приложение взаимодействует с базой данных PostgreSQL, размещенной в отдельном контейнере. Обмен данными между backend-контейнером и базой данных выполняется по протоколу TCP. PostgreSQL используется для хранения информации о пользователях, темах, идеях, комментариях, реакциях, избранных темах, достижениях, сообщениях поддержки и отозванных токенах безопасности. Для сохранения данных при перезапуске контейнеров используется постоянное хранилище.

Отдельным внешним сервисом выступает SMTP-сервер, применяемый для отправки писем подтверждения регистрации и восстановления пароля. Серверная часть приложения обращается к почтовому сервису через SMTP-соединение, используя параметры, заданные в переменных окружения. Это позволяет отделить механизм отправки сообщений от основной логики приложения и повысить гибкость настройки системы.

Взаимодействие компонентов организовано следующим образом. Пользователь или администратор отправляет запрос через браузер к клиентскому интерфейсу приложения. Клиентская часть обращается к backend API по протоколу HTTPS, передавая необходимые данные и JWT-токен авторизации. Серверная часть выполняет проверку прав доступа, обработку бизнес-логики и при необходимости обращается к базе данных PostgreSQL. При выполнении операций регистрации, подтверждения email или восстановления пароля backend дополнительно взаимодействует с SMTP-сервисом для отправки электронных сообщений.

Диаграмма также показывает разграничение ответственности между компонентами. Клиентская часть отвечает за отображение интерфейса и передачу запросов, backend-сервис — за выполнение бизнес-логики, проверку прав доступа и работу с данными. Такой подход исключает прямое обращение клиента к базе данных и повышает безопасность системы.

Применение Docker Compose обеспечивает изоляцию компонентов, упрощает настройку окружения и повышает переносимость приложения. Разделение системы на клиентскую часть, серверное приложение и базу данных соответствует трехзвенной архитектуре, способствует масштабируемости и упрощает сопровождение веб-приложения «IdeaFlow»

2.5 Проектирование базы данных

Логическая схема базы данных [23] – это визуальное представление структуры базы данных, показывающее таблицы, их атрибуты и взаимосвязи между ними. Логическая схема базы данных была выполнена с помощью сервиса draw.io [24] и представлена на рисунке 2.4 и в приложении Г.

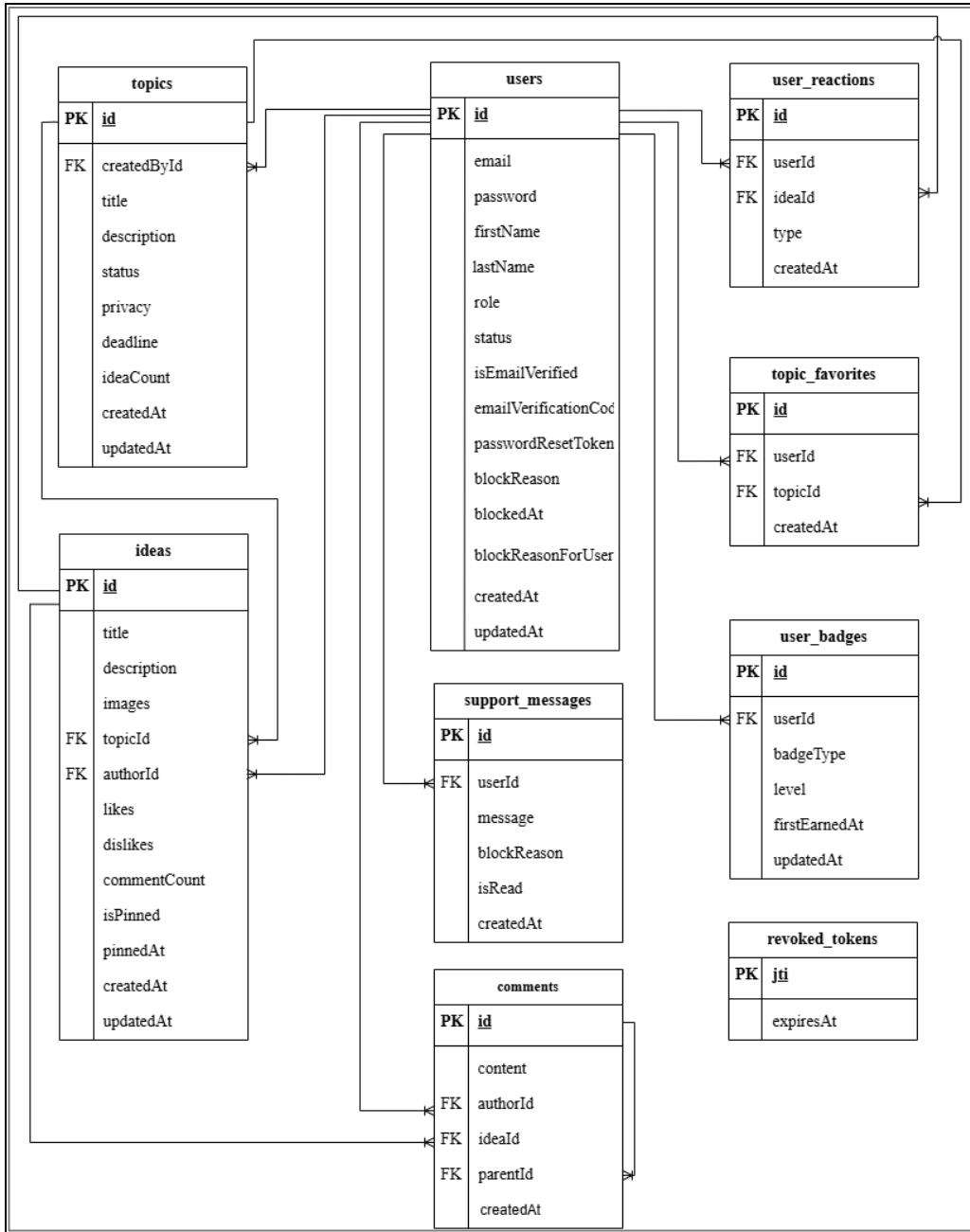


Рисунок 2.4. – Логическая схема базы данных

В базе данных платформы «IdeaFlow» содержится девять основных таблиц, обеспечивающих хранение и управление данными пользователей, тем, идей, комментариев, реакций, избранных тем, достижений, обращений в техническую поддержку и отозванных токенов безопасности. Большинство

таблиц используют тип данных UUID в качестве первичных ключей, что гарантирует уникальность идентификаторов и упрощает связывание записей между сущностями системы. Для части связей между таблицами применяется каскадное удаление зависимых записей, например для идей, связанных с темами, комментариев, вложенных комментариев и реакций на идеи. Такая модель данных логически разделяет основные сущности системы и снижает избыточность хранения информации.

Таблица users хранит информацию о зарегистрированных пользователях системы. Описание полей представлено в таблице 2.1.

Таблица 2.1 – Структура таблицы users

Название столбца	Тип данных	Описание
id	uuid	Идентификатор пользователя, первичный ключ
email	varchar	Электронная почта пользователя, используется как логин
password	varchar	Хешированный пароль пользователя
firstName	varchar	Имя пользователя
lastName	varchar	Фамилия пользователя
role	enum	Роль пользователя
status	enum	Статус пользователя
isEmailVerified	boolean	Признак подтверждения электронной почты
emailVerificationCode	varchar	Код подтверждения электронной почты
emailVerificationCodeExpires	timestamp	Срок действия кода подтверждения
passwordResetToken	varchar	Токен восстановления пароля
passwordResetTokenExpires	timestamp	Срок действия токена восстановления
blockReason	text	Причина блокировки для администратора
blockReasonForUser	text	Причина блокировки для пользователя
blockedAt	timestamp	Дата блокировки пользователя
createdAt	timestamp	Дата создания записи
updatedAt	timestamp	Дата последнего изменения записи

Таблица topics содержит сведения о темах обсуждений. Описание полей представлено в таблице 2.2.

Таблица 2.2 – Структура таблицы topics

Название столбца	Тип данных	Описание
1	2	3
id	uuid	Идентификатор темы, первичный ключ
title	varchar	Заголовок темы
description	text	Описание темы
status	enum	Состояние темы

Продолжение таблицы 2.2

1	2	3
privacy	enum	Уровень доступа
deadline	timestamp	Дата завершения темы
ideaCount	int	Количество идей в теме
createdById	uuid	Идентификатор автора темы
createdAt	timestamp	Дата создания темы
updatedAt	timestamp	Дата последнего изменения темы

Таблица ideas предназначена для хранения идей, предложенных пользователями. Описание полей представлено в таблице 2.3.

Таблица 2.3 – Структура таблицы ideas

Название столбца	Тип данных	Описание
id	uuid	Идентификатор идеи, первичный ключ
title	varchar	Заголовок идеи
images	simple-array	Список путей к прикрепленным изображениям
topicId	uuid	Внешний ключ на идентификатор темы
authorId	uuid	Внешний ключ на идентификатор автора идеи
likes	int	Количество положительных реакций
commentCount	int	Количество комментариев
isPinned	boolean	Признак закрепления идеи
createdAt	timestamp	Дата создания идеи
updatedAt	timestamp	Дата последнего изменения идеи

Таблица comments реализует систему комментариев. Описание полей представлено в таблице 2.4.

Таблица 2.4 – Структура таблицы comments

Название столбца	Тип данных	Описание
id	uuid	Идентификатор комментария, первичный ключ
content	text	Текст комментария
ideaId	uuid	Внешний ключ на идентификатор идеи
authorId	uuid	Внешний ключ на идентификатор автора комментария
parentId	uuid	Идентификатор родительского комментария, если комментарий является ответом
createdAt	timestamp	Дата создания комментария

Таблица user_reactions хранит реакции пользователей на идеи. Это позволяет анализировать пользовательский интерес к опубликованным предложениям и определять наиболее востребованные идеи. Описание полей представлено в таблице 2.5.

Таблица 2.5 – Структура таблицы user_reactions.

Название столбца	Тип данных	Описание
id	uuid	Идентификатор реакции, первичный ключ
userId	uuid	Внешний ключ на идентификатор пользователя
ideaId	uuid	Внешний ключ на идентификатор идеи
createdAt	timestamp	Дата создания реакции

Таблица topic_favorites содержит информацию об избранных темах пользователей. Описание полей представлено в таблице 2.6.

Таблица 2.6 – Структура таблицы topic_favorites

Название столбца	Тип данных	Описание
id	uuid	Идентификатор записи, первичный ключ
userId	uuid	Внешний ключ на идентификатор пользователя
topicId	uuid	Внешний ключ на идентификатор темы

Таблица user_badges хранит сведения о достижениях пользователей. Описание полей представлено в таблице 2.7.

Таблица 2.7 – Структура таблицы user_badges

Название столбца	Тип данных	Описание
id	uuid	Идентификатор достижения, первичный ключ
userId	uuid	Внешний ключ на идентификатор пользователя
badgeType	varchar	Тип достижения
level	int	Уровень достижения
firstEarnedAt	timestamp	Дата первого получения достижения
updatedAt	timestamp	Дата последнего обновления достижения

Таблица support_messages содержит обращения пользователей в техническую поддержку. Описание полей представлено в таблице 2.8.

Таблица 2.8 – Структура таблицы support_messages

Название столбца	Тип данных	Описание
1	2	3
id	uuid	Идентификатор сообщения, первичный ключ
userId	uuid	Внешний ключ на идентификатор пользователя
message	text	Текст обращения в техническую поддержку

Продолжение таблицы 2.8

1	2	3
blockReason	text	Причина блокировки, если обращение связано с блокировкой
isRead	boolean	Признак прочтения сообщения администратором
createdAt	timestamp	Дата создания сообщения

Таблица `revoked_tokens` хранит отозванные JWT-токены. Описание полей представлено в таблице 2.9.

Таблица 2.9 – Структура таблицы `revoked_tokens`

Название столбца	Тип данных	Описание
id	uuid	Идентификатор записи, первичный ключ
token	text	Отозванный JWT-токен
expiresAt	timestamp	Срок действия токена

Связи между таблицами отражают основные функциональные требования системы. Один пользователь может создавать несколько тем, идей, комментариев, реакций, избранных тем, достижений и обращений в техническую поддержку. Одна тема может содержать множество идей и добавляться в избранное разными пользователями. Каждая идея может иметь несколько комментариев и реакций, а комментарии могут образовывать иерархическую структуру за счет связи с родительскими комментариями.

2.6 Контейнеризация

Для обеспечения изоляции, воспроизводимости и управляемости компонентов веб-приложения IdeaFlow была применена методология контейнеризации на базе платформы Docker. Архитектура системы реализована в виде трех автономных, но функционально связанных контейнеров: `ideaflow_backend` (серверная часть на базе Nest.js), `ideaflow_frontend` (клиентский интерфейс) и `ideaflow_db` (СУБД PostgreSQL). Данное разделение соответствует принципам микросервисной архитектуры и способствует соблюдению границ ответственности между слоями приложения в процессе его развертывания.

Оркестрация контейнеров осуществляется с использованием Docker Compose [26], что обеспечивает декларативное описание инфраструктуры, автоматизацию сетевого взаимодействия сервисов и централизованное управление переменными окружения. Контейнеризация позволила достичь полной независимости сред выполнения, гарантировать идентичность конфигурации на всех этапах жизненного цикла ПО и упростить процессы развертывания и масштабирования системы в соответствии с проектными требованиями.

Docker Compose задаёт порядок запуска сервисов и обеспечивает подключение контейнеров к общей внутренней сети. Это упрощает обмен данными между компонентами и снижает риск ошибок конфигурации.

2.7 Проектирование основных алгоритмов

Проектирование ключевых алгоритмов является важным этапом разработки, предшествующим непосредственной реализации программной системы. Четкое описание логики выполнения основных бизнес-процессов обеспечивает правильное понимание функциональных требований и служит основой для последующего написания и тестирования программного кода.

Алгоритм описывает последовательность действий пользователя при создании новой темы для обсуждения. Начальным условием является успешная аутентификация и наличие прав на отправку темы на рассмотрение.

Процесс начинается с открытия формы. Пользователь заполняет обязательные поля, после чего выполняется первичная проверка данных на стороне клиента. При успешной проверке данные отправляются на сервер.

На сервере проверяются JWT-токен, права доступа и корректность данных через DTO [27]. При успешной валидации тема сохраняется в базе данных со статусом PENDING, так как пользовательские темы требуют рассмотрения администратором. После этого сервер возвращает подтверждение, а клиент отображает уведомление об успешной отправке темы на модерацию. При возникновении ошибки пользователю выводится соответствующее сообщение. Алгоритм создания пользователем новой темы представлен в приложении Д.

Таким образом, алгоритм создания новой темы обеспечивает разграничение действий в зависимости от роли пользователя и позволяет реализовать контролируемый процесс добавления контента. Использование проверки прав доступа, валидации данных и автоматического назначения статуса темы повышает безопасность системы и снижает вероятность появления некорректных или несанкционированных данных. Описанный алгоритм служит основой для реализации соответствующей серверной логики и пользовательского интерфейса веб-приложения «IdeaFlow».

2.7 Выводы по разделу

В данном разделе представлена архитектура веб-приложения «IdeaFlow», построенная по трехзвенной модели. Спроектированы реляционная база данных с основными сущностями системы и схема развертывания на основе Docker. Определен технологический стек проекта: React.js для клиентской части, Nest.js и TypeORM для серверной части, PostgreSQL в качестве СУБД, а также JWT, bcrypt и Nodemailer для реализации безопасности и почтовых уведомлений. Также рассмотрены роли пользователей, основные варианты использования и ключевые алгоритмы, включая создание новой темы. Это позволило определить границы ответственности ролей, обеспечить соблюдение принципа минимальных привилегий и подготовить систему к программной реализации. Таким образом, результаты проектирования формируют целостное представление о структуре приложения, взаимодействии его компонентов и логике выполнения основных бизнес-процессов.

3 Разработка веб-приложения

Веб-приложение будет разрабатываться с использованием стека современных технологий. Серверная часть реализуется на языке TypeScript с применением фреймворка Nest.js, что обеспечивает структурированную, модульную и масштабируемую архитектуру. Для построения динамического и отзывчивого клиентского интерфейса будет использоваться библиотека ReactJS. Взаимодействие с данными будет организовано через TypeORM – мощную ORM-систему для TypeScript и JavaScript, которая обеспечивает интеграцию с базами данных и полностью соответствует типобезопасной экосистеме используемого в проекте стека технологий.

3.1 Разработка серверной части

Серверная часть платформы IdeaFlow была разработана с использованием современного фреймворка NestJS, который предоставляет структурированный подход к построению масштабируемых серверных приложений на Node.js. Основное внимание при разработке было уделено созданию надежной и безопасной системы аутентификации и авторизации пользователей. Для этого была реализована JWT (JSON Веб Token) аутентификация с использованием библиотеки Passport.js, что обеспечивает безопасный доступ к защищенным ресурсам. Система включает многоуровневую проверку пользователей: регистрацию с подтверждением email, вход с валидацией учетных данных, восстановление пароля через email, а также механизм блокировки и разблокировки аккаунтов администратором.

Архитектура серверной части построена по модульному принципу, где каждый функциональный блок (авторизация, управление пользователями, темы, идеи, достижения) выделен в отдельный модуль с собственной бизнес-логикой. Для работы с базой данных используется ORM TypeORM в связке с PostgreSQL, что обеспечивает типобезопасность и удобство работы с сущностями. Были созданы модели данных для пользователей, тем обсуждения, идей, комментариев, реакций, избранного, достижений и сообщений поддержки. Особое внимание уделено отношениям между сущностями: пользователь может создавать темы, предлагать идеи в рамках тем, комментировать идеи, выражать реакции через систему реакций, добавлять темы в избранное и получать достижения за активность. В процессе разработки серверной части также была реализована единая структура обработки запросов и ответов API. Контроллеры принимают входные данные через DTO, после чего передают их в сервисы, где выполняется основная бизнес-логика и взаимодействие с базой

					ДП 03.00.ПЗ		
		ФИО	Подпись	Дата	3 Разработка веб-приложения		
Разраб.		Телица А.Д.					
Пров.		Сазонова Д.В.					
Н. контр.		Николайчук А.Н.					
Утв.		Блинова Е.А.					
					Лит.	Лист	Листов
					у	1	21
					БГТУ 1-40 05 01, 2026		

данных. Такой подход позволил отделить транспортный уровень от логики приложения, повысить читаемость кода и упростить дальнейшее расширение функциональности системы

Для обеспечения безопасности реализована система ролей (администратор, пользователь и гость) с гибкой системой прав доступа. Администраторы получают расширенные возможности: просмотр всех пользователей системы, блокировка аккаунтов с указанием причин, управление контентом, модерация тем. Все конечные точки API защищены Guard-ами[20], которые проверяют наличие действительного JWT-токена и соответствие роли пользователя требуемым правам доступа. Дополнительно реализован механизм отзыва токенов при выходе из системы, что предотвращает использование скомпрометированных токенов. На рисунке 3.1 представлена структура серверной части, демонстрирующая архитектуру серверной логики.

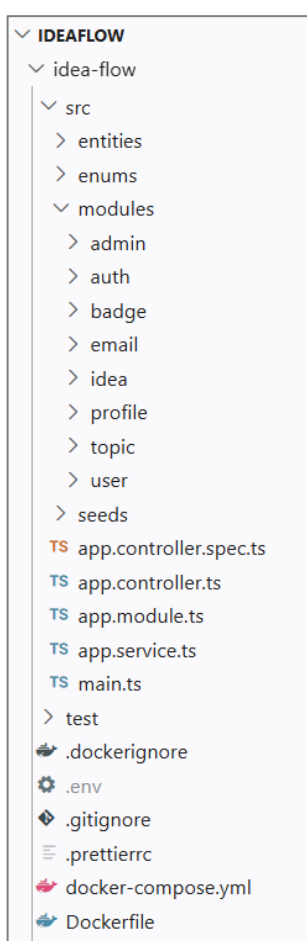


Рисунок 3.1 – Структура серверной части

Модуль авторизации представляет собой автономный компонент системы, спроектированный для обеспечения идентификации, аутентификации и управления доступом пользователей к защищенным ресурсам приложения. Его архитектура включает контроллер для обработки HTTP-запросов и сервис, инкапсулирующий бизнес-логику, такую как регистрация, вход в систему, восстановление пароля и верификация электронной почты, используя соответ-

ствующие объекты передачи данных (DTO). Стратегия JWT совместно с защитными механизмами (Guards) обеспечивает валидацию токенов и аутентификацию на уровне конечных точек, тогда как Roles Guard и Roles Decorator реализуют ролевую модель контроля доступа. Присутствие декоратора упрощает извлечение аутентифицированных данных пользователя в обработчиках маршрутов, формируя таким образом комплексное и структурированное решение для управления сессиями и безопасностью.

Модуль администрирования предоставляет администраторам централизованное управление учетными записями и обращениями в поддержку. Доступ к его функционалу ограничен ролью ADMIN через JWT и ролевые гарды. Контроллер предоставляет API для операций с пользователями: блокировка с указанием причины, разблокировка пользователей, просмотр статистики активности и взаимодействие с сообщениями службы поддержки. В свою очередь сервис обеспечивает работу с данными через TypeORM.

Модуль электронной почты отвечает за асинхронную отправку уведомлений через SMTP [28] с использованием Nodemailer [29]. Он работает независимо от основного приложения, получая конфигурацию из переменных среды. Сервис модуля инициализирует и проверяет соединение, а также предоставляет методы для отправки ключевых писем: подтверждения регистрации, сброса пароля и уведомлений о блокировке учетной записи.

Модуль профиля реализует управление персональными данными в условиях аутентифицированного доступа с использованием JWT-токенов и механизма охраны доступа. Его зависимость от модуля авторизации обеспечивает строгую верификацию пользователя. Для оптимизации производительности применяется кастомный декоратор @GetUser, который предоставляет данные текущего пользователя без дополнительных запросов к базе данных.

Модуль управления темами отвечает за создание и управление темами в системе краудсорсинга. Он разделяет права на их создание: администраторы публикуют темы сразу, а обычные пользователи отправляют предложения на предварительное рассмотрение. Контроллер предоставляет полный набор CRUD-операций для работы с темами, включая функции модерации (одобрение/отклонение), а также управление избранным темами.

Модуль идей обеспечивает платформу для создания, обсуждения и оценки предложений пользователей в рамках утвержденных тем. Он реализует полный цикл CRUD-операций, а также включает механизмы: комментариев с иерархическими ответами, рейтинговую систему реакций идей, закрепление и открепление идей. Контроллер предоставляет также аналитический эндпоинт [30] для получения статистики по активности пользователя.

Модуль достижений реализует систему геймификации для повышения вовлеченности пользователей. Сервис автоматически отслеживает активность пользователей (количество идей, комментариев, полученных реакций, созданных тем) и присваивает достижения различных уровней при достижении пороговых значений. Поддерживаются достижения: «Первый автор», «Гуру комментариев», «Самый популярный автор», «100 реакций» и «Мастер тем».

3.1.1 Реализация модуля авторизации

Модуль авторизации реализует аутентификацию и разграничение доступа на основе JSON Веб Token (JWT) в NestJS. JWT обеспечивает stateless-аутентификацию [31] без хранения сессий на сервере, после входа токен передается клиенту и используется в заголовке Authorization.

Регистрация выполняется через метод register. Перед созданием учетной записи данные валидируются с помощью class-validator [32] на основе RegisterDto, который проверяет формат email, сложность пароля и корректность имени. При успешной валидации выполняется проверка существующего пользователя, пароль хешируется с помощью bcrypt, генерируется 6-значный верификационный код с временем жизни 1 час, после чего создается запись пользователя в базе данных через TypeORM с начальным статусом PENDING. После сохранения пользователя система отправляет письмо с кодом верификации через EmailModule, что обеспечивает подтверждение email. Код метода представлен на листинге 3.1.

```

    async register(registerDto: RegisterDto): Promise<{ message:
string }> {
        const { email, password, firstName, lastName } = regis-
terDto;
        const existingUser = await this.userRepository.findOne({
where: { email } });
        if (existingUser && existingUser.isEmailVerified) {
            throw new ConflictException('Пользователь с такой почтой
уже существует');
        }
        const hashedPassword = await bcrypt.hash(password, 12);
        const verificationCode = this.generateVerificationCode();
        const verificationCodeExpires = new Date(Date.now() + 60 *
60 * 1000);
        const user = this.userRepository.create({
            email,
            password: hashedPassword,
            firstName,
            lastName,
            status: UserStatus.PENDING,
            emailVerificationCode: verificationCode,
            emailVerificationCodeExpires,
            isEmailVerified: false,});
        await this.userRepository.save(user);
        await this.emailService.sendVerificationEmail(email, veri-
ficationCode);
        return {
            message: 'Регистрация прошла успешно. Проверьте почту для
получения кода подтверждения.',
        };
    }

```

Листинг 3.1 – Реализация метода регистрации пользователей

Аутентификация пользователей выполняется методом `login`. Он проверяет наличие пользователя по email, валидирует пароль через `bcryptjs.compare`, а также контролирует статусы аккаунта: блокировку и неподтвержденный email. При успешной проверке генерируется JWT-токен через `jwtService.sign`. В payload токена включаются идентификатор пользователя, email, роль, флаг верификации и уникальный идентификатор JWT. Срок жизни токена устанавливается в 24 часа. Код метода `login` представлен на листинге 3.2.

```

    async login(loginDto: LoginDto): Promise<{ access_token:
string }> {
    const { email, password } = loginDto;
    const user = await this.userRepository.findOne({ where: {
email } });
    if (!user) {
        throw new UnauthorizedException('Неверные данные');
    }

    if (user.status === UserStatus.BLOCKED) {
        throw new ForbiddenException('Ваш аккаунт был заблокиро-
ван.');
```

```

    }

    if (!user.isEmailVerified) {
        throw new ForbiddenException('Пожалуйста подтвердите ваш
email');
```

```

    }

    const isPasswordValid = await bcrypt.compare(password,
user.password);
    if (!isPasswordValid) {
        throw new UnauthorizedException('Неверные данные');
    }
    const payload = {
        sub: user.id,
        email: user.email,
        role: user.role,
        verified: user.isEmailVerified,
        jti: uuidv4(),
    };
    return {
        access_token: this.jwtService.sign(payload),
    };
}

```

Листинг 3.2 – Реализация авторизации пользователей

Верификация email выполняется методом `verifyEmail`. Пользователь предоставляет email и шестизначный код, который сверяется с данными в базе. При совпадении и актуальности кода статус меняется на `ACTIVE`, флаг `isEmailVerified` устанавливается в `true`, после чего генерируется JWT. Также предусмотрена повторная отправка кода.

Выход из системы реализован посредством механизма отзыва токена. JWT-токен извлекается из заголовка запроса, декодируется для получения уникального `jwt` и его значение сохраняется в таблице `RevokedToken`. При каждом запросе `JwtStrategy` проверяет, не был ли токен отозван.

Архитектура модуля гарантирует безопасность за счет хеширования паролей и аутентификации по JWT. Применяются строгие проверки статуса, верификации email и механизм отзыва токенов для немедленного завершения сессии. Модульная структура способствует повторному использованию логики и механизмов защиты.

3.1.2 Реализация модуля управления темами для обсуждения

Модуль управления темами обеспечивает создание, модерацию и управление тематическими обсуждениями. Основная логика модуля основана на разграничении прав доступа: администратор может публиковать темы напрямую, а пользователь – только отправлять их на рассмотрение. Контроллер предоставляет защищенные REST API-маршруты с использованием гардов. Для администратора применяется метод `create`, который создает тему сразу в статусе `APPROVED`. Код метода представлен на листинге 3.3.

```
@Post()
@UseGuards(JwtAuthGuard, RolesGuard)
@Roles(UserRole.ADMIN)
create(@Body() createTopicDto: CreateTopicDto, @GetUser()
user: User) {
    return this.topicService.create(createTopicDto, user);
}
async create(createTopicDto: CreateTopicDto, user: User):
Promise<any> {

    if (!user.isEmailVerified) {
        throw new BadRequestException(
            'Для создания тем необходимо подтвердить email',
        );
    }

    const topic = this.topicRepository.create({
        ...createTopicDto,
        createdBy: user,
        createdById: user.id,
        status: TopicStatus.APPROVED,
    });
    return this.topicRepository.save(topic);
}
```

Листинг 3.3 – Метод и маршрут `create` для администратора

В то время как обычные пользователи могут лишь предлагать темы для рассмотрения через защищенный маршрут `suggest`, администраторы могут создавать темы напрямую в статусе `APPROVED` через метод `create`.

После создания тема попадает в один из четырех статусов: PENDING (ожидает), APPROVED (одобрена), REJECTED (отклонена) или CLOSED (закрыта). Для управления этим циклом администраторы используют методы `approve` и `reject`, которые меняют статус темы после соответствующих проверок. При одобрении темы также вызывается сервис начисления достижений (`badgeService`). Код метода `approve` представлен на листинге 3.4.

```

async approve(id: string): Promise<any> {
    const topic = await this.topicRepository.findOne({
        where: { id },
        relations: ['createdBy'],    });
    if (!topic) {
        throw new NotFoundException('Topic not found');
    }
    if (topic.status === TopicStatus.APPROVED) {
        throw new BadRequestException('Topic is already approved');
    }
    topic.status = TopicStatus.APPROVED;
    const savedTopic = await this.topicRepository.save(topic);
    return this.formatTopicResponse(savedTopic, true);
}

```

Листинг 3.4 – Одобрение предложенной темы администратором

Также администраторам доступно управление темами на любом из этапов жизненного цикла. Механизм взаимодействия с идеями реализован аналогично, за исключением процесса одобрения администратором. Идеи добавляются к уже одобренным темам без дополнительной модерации, при условии, что срок подачи идей не истек.

3.1.3 Реализация механизма блокировки пользователей

Система реализует процедуру блокировки пользователей через метод `blockUser`, который включает многоуровневую валидацию и проверку условий выполнения операции. При получении запроса на блокировку система выполняет поиск пользователя по идентификатору, проверяет его роль, так как администраторы не могут быть заблокированы, и текущий статус. В случае успешных проверок пользователю присваивается статус `BLOCKED`, фиксируются причина блокировки и временная метка. Система поддерживает разделение причин: внутреннюю для администраторов (`blockReason`) и ту, что отображается пользователю (`blockReasonForUser`), хотя в текущей реализации оба поля заполняются одинаковым значением. Такой подход позволяет сохранять историю административного воздействия и делает процесс модерации более прозрачным и управляемым.

Заблокированный пользователь получает возможность инициировать обращение в службу поддержки через метод `sendSupportMessage`, который реали-

зует контролируемый канал связи. Алгоритм предусматривает верификацию статуса пользователя и создание структурированной записи обращения в базе данных. Это позволяет пользователю получить разъяснение причины блокировки или отправить запрос на пересмотр решения без доступа к основным функциям платформы. Код метода `sendSupportMessage` представлен на листинге 3.5.

```
async sendSupportMessage(email: string, message: string, blockReason?: string): Promise<{ message: string }> {
  const user = await this.userRepository.findOne({ where: {
    email } });

  if (!user) {
    throw new NotFoundException('Пользователь не найден');
  }

  if (user.status !== UserStatus.BLOCKED) {
    throw new BadRequestException('Только заблокированные пользователи могут отправлять сообщения в службу поддержки');
  }

  const supportMessage = this.supportMessageRepository.create({
    userId: user.id,
    message,
    blockReason: blockReason || user.blockReason,
  });
  await this.supportMessageRepository.save(supportMessage);

  return { message: 'Сообщение службы поддержки успешно отправлено' };
}
```

Листинг 3.5 – Обращение заблокированных пользователей

Интерфейс администратора предоставляет инструменты для мониторинга обращений через метод `getAllSupportMessages`, который формирует выборку сообщений с присоединенными данными пользователей. Для каждого обращения система отображает электронную почту, имя, текст сообщения, причину блокировки и статус прочтения. Процедура разблокировки реализована в методе `unlockUser`: администратор может снять блокировку, после чего система обнуляет причину блокировки, временные метки и переводит статус пользователя в `ACTIVE`. Для отметки сообщений поддержки как прочитанных используется отдельный метод `markSupportMessageAsRead`.

3.1.4 Реализация подтверждения email и восстановления пароля

Сервис электронной почты обеспечивает отправку писем через SMTP-сервер с использованием библиотеки `Nodemailer`. При запуске создается почтовый транспортер, параметры которого берутся из переменных окружения, после чего выполняется проверка соединения с SMTP-сервером.

В сервисе реализованы методы `sendVerificationEmail()` для подтверждения регистрации и `sendPasswordResetEmail()` для восстановления пароля. Они формируют HTML-письма с кодом подтверждения или токеном сброса и отправляют их через Nodemailer.

3.1.5 Реализация механизма реакций

Система реакций реализована как часть сервиса идей и использует сущность `UserReaction` для хранения взаимодействий пользователей с контентом. Модель предусматривает одно состояние – `LIKE`, отражающее положительную оценку идеи. Перед сохранением реакции система проверяет наличие уже существующей записи для пары пользователь-идея, что предотвращает дублирование лайков. Метод `like()` отвечает за добавление или отмену положительной реакции, обновление счетчика в сущности `Idea` и пересчет достижений автора идеи. Код метода представлен на листинге 3.6.

```
async like(id: string, user: User): Promise<any> {
  if (!user.isEmailVerified) {
    throw new BadRequestException('Необходимо подтвердить email');
  }
  const idea = await this.ideaRepository.findOne({ where: { id } });
  if (!idea) throw new NotFoundException('Идея не найдена');
  const reaction = await this.userReactionRepository.findOne({
    where: { userId: user.id, ideaId: id },
  });
  if (reaction) {
    await this.userReactionRepository.remove(reaction);
    idea.likes -= 1;
  } else {
    await this.userReactionRepository.save({
      userId: user.id,
      ideaId: id,
      type: ReactionType.LIKE,
    });
    idea.likes += 1;
  }
  return this.ideaRepository.save(idea);
}
```

Листинг 3.6 – Метод положительной реакции для идеи

Основной алгоритм обработки реакции включает проверку подтверждения email пользователя, существования идеи и доступности связанной с ней темы. Если пользователь еще не ставил лайк, создается новая запись в таблице `UserReaction`, а счетчик `likes` увеличивается на единицу. При повторном нажатии существующая реакция удаляется, а счетчик уменьшается. Такой подход

позволяет реализовать переключаемую реакцию без создания дубликатов и сохраняет целостность данных.

Также в проекте реализован механизм инициализации привилегированного пользователя при запуске приложения. Модуль AdminSeed автоматически создает системную учетную запись администратора при первом запуске. Листинг кода сидера представлен в приложении И. В главный модуль приложения подключены все основные функциональные модули системы.

3.1.6 Реализация системы вложенных комментариев

Для поддержки структурированного обсуждения идей реализована система вложенных комментариев, позволяющая пользователям отвечать на конкретные сообщения, формируя древовидную иерархию диалогов. Архитектурно это реализовано через самоссылочную связь сущности Comment.

Сущность Comment содержит два ключевых поля для организации иерархии: parent – связь многие к одному с родительским комментарием, и replies – коллекция один ко многим дочерних ответов. Поле parentId хранит идентификатор родительского комментария, что позволяет создавать произвольную вложенность обсуждений и связывать ответы с исходными сообщениями. На листинге 3.7 представлен фрагмент реализации сущности Comment с самоссылочной связью.

```
@ManyToOne(() => Comment, { nullable: true, onDelete: 'CASCADE' })
@JoinColumn({ name: 'parentId' })
parent: Comment;

@Column({ nullable: true })
parentId: string;

@OneToMany(() => Comment, comment => comment.parent)
replies: Comment[];
```

Листинг 3.7 – Поля сущности Comment с самоссылочной связью

При создании комментария через DTO передается опциональный параметр parentId, указывающий на родительский комментарий. Метод addComment сервиса сохраняет новый комментарий вместе со ссылкой на родителя, если пользователь отвечает на уже существующее сообщение. При отсутствии parentId комментарий считается корневым и отображается на верхнем уровне обсуждения. При получении списка комментариев поле parentId включается в ответ, что позволяет клиентской части корректно восстановить вложенную структуру и отобразить ответы под соответствующими сообщениями. Такой подход упрощает навигацию по обсуждению и делает диалог между пользователями более наглядным. Это также снижает сложность обработки комментариев на клиентской стороне, поскольку каж-

дая запись уже содержит информацию о своей позиции в иерархии обсуждения. На листинге 3.8 представлен метод добавления комментария с поддержкой вложенности сообщений.

```
async addComment(
  id: string,
  createCommentDto: CreateCommentDto,
  user: User,
): Promise<any> {
  if (!user.isEmailVerified) {
    throw new BadRequestException(
      'Для добавления комментариев необходимо подтвердить email',
    );
  }

  const idea = await this.ideaRepository.findOne({ where: { id } });
  if (!idea) throw new NotFoundException('Идея не найдена');

  const comment = this.commentRepository.create({
    content: createCommentDto.content,
    parentId: createCommentDto.parentId,
    ideaId: idea.id,
    authorId: user.id,
  });

  return this.commentRepository.save(comment);
}
```

Листинг 3.8 – Добавление вложенного комментария

Использование декоратора позволяет создавать как корневые комментарии, так и вложенные ответы. Опция каскадного удаления обеспечивает автоматическое удаление всех дочерних комментариев при удалении родительского, поддерживая целостность иерархии обсуждений.

3.1.7 Система достижений и бейджей

Для стимулирования активности пользователей реализована система достижений, которая автоматически начисляет награды за определенные действия в системе. Архитектура построена на событийной модели: BadgeService отслеживает действия пользователя (публикация идей, комментариев, получение реакций, создание тем) и пересчитывает соответствующие достижения.

Система поддерживает пять типов достижений:

- FIRST_AUTHOR – за публикацию первой идеи;
- COMMENT_GURU – за количество комментариев;
- POPULAR_AUTHOR – за получение максимального количества положительных реакций на одной опубликованной идее;
- TOTAL_LIKES – за суммарное количество реакций на всех идеях;
- TOPIC_MASTER – за создание 5 одобренных тем;

- EDITORS_CHOICE — за 3 закрепленные идеи;
- IDEA_GENERATOR — за публикацию 10 идей;
- COMMUNITY_HEART — за постановку 10 положительных реакций идеям других пользователей;
- KNOW_IT_ALL — за публикацию комментариев в 5+ разных темах.

На листинге 3.9 представлен сервис BadgeService с логикой автоматического начисления достижений.

```
const COMMENT_THRESHOLDS = [10, 50, 100] as const;
@Injectable()
export class BadgeService {
  async evaluateAfterCommentAdded(userId: string): Promise<void> {
    const count = await this.commentRepository.count({
      where: { authorId: userId },
    });

    const level = this.levelFromThresholds(count, COMMENT_THRESHOLDS);
    if (level <= 0) return;

    const badge = await this.userBadgeRepository.findOne({
      where: { userId, badgeType: BadgeType.COMMENT_GURU },
    });

    await this.userBadgeRepository.save({
      ...badge,
      userId,
      badgeType: BadgeType.COMMENT_GURU,
      level: Math.max(level, badge?.level ?? 0),
      firstEarnedAt: badge?.firstEarnedAt ?? new Date(),
    });
  }

  private levelFromThresholds(value: number, thresholds:
readonly number[]): number {
    return thresholds.filter(threshold => value >= threshold).length;
  }
}
```

Листинг 3.9 – Сервис автоматического начисления достижений

Система использует пороговые значения для определения уровней достижений. Методы `evaluateAfter` вызываются из соответствующих сервисов после выполнения пользователем целевых действий, что обеспечивает реактивное обновление достижений без необходимости ручного пересчета. Это позволяет автоматически фиксировать прогресс пользователя и своевременно присваивать новые достижения при изменении его активности.

3.2. Разработка клиентской части

Клиентская часть системы представляет собой веб-приложение, разработанное на базе реактивного фреймворка React версии 18.2.0 в сочетании со строго типизированным языком программирования TypeScript 5.3.0. Выбор этого стека обеспечивает создание высокопроизводительного и поддерживаемого интерфейса. Использование статической типизации TypeScript позволяет выявлять ошибки, связанные с типами данных, на этапе компиляции, что повышает надежность кода, упрощает рефакторинг и способствует долгосрочной масштабируемости приложения по мере роста его функциональности. Компонентный подход React позволяет разделять интерфейс на независимые переиспользуемые элементы, что упрощает сопровождение клиентской части и ускоряет добавление новых функций.

В качестве инструмента для инициализации проекта и управления процессом сборки был задействован Create React App 5.0.1 с react-scripts. Для реализации навигации внутри SPA без перезагрузки страниц использована библиотека React Router DOM 6.20.0, которая обеспечивает декларативную маршрутизацию и вложенные макеты страниц. Такой подход позволяет организовывать переходы между основными разделами приложения, сохраняя целостность пользовательского опыта и избегая полной перезагрузки страницы.

Организация асинхронного взаимодействия с серверным RESTful API осуществляется через библиотеку Axios 1.6.0. Ее преимуществами являются удобный API для работы с HTTP-запросами, встроенная поддержка перехватчиков для централизованной обработки запросов и ответов, а также возможность отмены запросов при изменении состояния интерфейса или переходе между страницами. Это особенно важно для динамических экранов, где данные обновляются после действий пользователя, например при добавлении идей, комментариев или реакций.

Для обработки пользовательского ввода применен подход с использованием библиотеки React Hook Form 7.49.2 в сочетании со схемами валидации на основе Yup 1.3.0. React Hook Form оптимизирует производительность форм за счет минимального количества ререндеров, а Yup предоставляет удобный синтаксис для декларативного описания сложных правил валидации, которые затем интегрируются с формами и обеспечивают единообразную проверку вводимых данных. Данный подход используется в формах регистрации, авторизации, создания тем, публикации идей и отправки обращений в поддержку.

Для поддержания кроссплатформенной консистентности переменных окружения в скриптах сборки используется утилита cross-env 7.0.3 [34]. Она позволяет одинаково задавать переменные окружения в разных операционных системах, что упрощает запуск и сборку проекта в локальной среде и при развертывании. Дополнительно клиентская часть обеспечивает взаимодействие пользователя с основными функциями платформы: просмотром тем, публикацией идей, добавлением комментариев, работой с избранным и отображением статистики активности. За счет разделения интерфейса на страницы, компоненты и сервисы обеспечивается удобная организация кода и возможность

дальнейшего расширения функциональности. Файловая структура проекта для клиентской части, иллюстрирующая описанную организацию, представлена на рисунке 3.2.

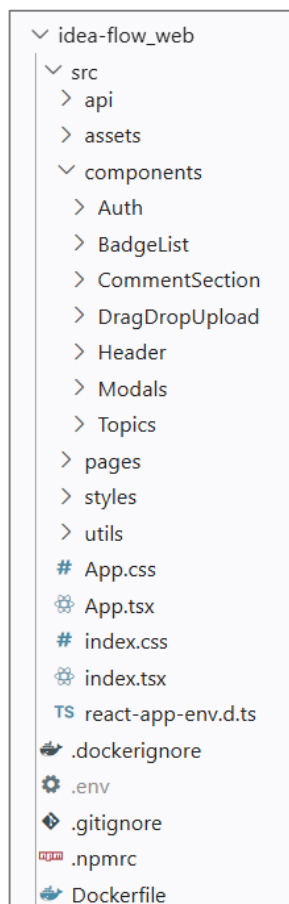


Рисунок 3.2 – Файловая структура проекта IdeaFlow-веб

Таким образом, применение модульной архитектуры и современных инструментов разработки позволило создать структурированный, производительный и удобный для пользователя интерфейс, готовый к дальнейшему масштабированию и поддержке.

3.2.1 Организация модулей представления

Основные пользовательские интерфейсы, соответствующие маршрутам приложения, сгруппированы в каталоге `pages`. Каждый модуль является самостоятельным компонентом, отвечающим за рендеринг полноценного экрана. Такая организация упрощает навигацию по приложению и позволяет разделять логику отображения разных функциональных областей.

Модуль `Auth` содержит страницы, реализующие основные сценарии аутентификации пользователей: авторизацию, регистрацию с подтверждением email и восстановление пароля. Дополнительно управление доступом к защищенным разделам приложения реализуется через компонент `PrivateRoute`:

- LoginPage.tsx – интерфейс ввода учетных данных для авторизации зарегистрированных пользователей;
- RegisterPage.tsx – форма регистрации нового пользователя;
- ForgotPasswordPage.tsx – интерфейс восстановления пароля через email с вводом токена и нового пароля.

Модуль Dashboard представляет собой аналитическую панель, визуализирующую ключевые метрики и данные системы. Он предназначен для администраторов и позволяет в реальном времени получать обзор активности пользователей, количества опубликованных идей, комментариев и прочих статистических показателей в реальном времени.

Модуль Home является точкой входа и главной страницей приложения, выполняющей роль навигационного хаба. На ней отображаются доступные темы, предлагается быстрый доступ к основным функциям и обеспечивается удобная ориентация пользователя в пространстве платформы.

Модуль TopicDetail обеспечивает детализированное представление и взаимодействие с отдельной тематической единицей контента. В рамках этого модуля пользователи могут просматривать идеи, добавлять комментарии, ставить реакции и управлять избранным.

Модуль UserDashboard реализует функциональность персональную панель управления пользователя, а также отображение главной страницы у пользователя. Здесь пользователь может просматривать свои идеи, комментарии, сводную статистику активности и управлять настройками профиля.

3.2.2 Организация асинхронного взаимодействия с сервером

Для обеспечения надежного и структурированного взаимодействия клиентского приложения с серверной частью была разработана централизованная система работы с HTTP-запросами. Основой данной системы стал модуль `api/index.ts`, который инкапсулирует настройку HTTP-клиента, обработку заголовков авторизации, перехватчики для глобальной обработки ответов и ошибок, а также предоставляет удобный объектно-ориентированный интерфейс для вызова всех конечных точек серверного API.

Реализация модуля API основана на нескольких ключевых принципах: базовая конфигурация `Axios` с настройкой URL и таймаутов; перехватчик запросов для автоматического добавления JWT-токена в заголовки; перехватчик ответов для централизованной обработки ошибок авторизации и сетевых сбоев; а также модульная структура, группирующая методы по доменным областям. Такой подход позволяет отделить логику работы с HTTP от компонентов пользовательского интерфейса, упрощает тестирование и обеспечивает единообразную обработку всех запросов и ответов в приложении. Благодаря централизованной настройке перехватчиков разработчики могут сосредоточиться на бизнес-логике, не дублируя код авторизации и обработки ошибок в каждом компоненте. Кроме того, единая точка конфигурации упрощает адаптацию приложения к изменениям в серверном API и позволяет быстро вносить

правки в параметры запросов без необходимости изменения множества файлов. Фрагмент кода реализации интерсепторов для организации HTTP-запросов представлен на листинге 3.7.

```
api.interceptors.request.use(
  (config) => {
    const token = localStorage.getItem('access_token');
    if (token) {
      config.headers.Authorization = Bearer ${token};
    }
    return config;
  },
  (error) => {
    return Promise.reject(error);
  }
);
api.interceptors.response.use(
  (response) => response,
  async (error) => {
    const originalRequest = error.config;
    if (error.response?.status === 401 && !originalRequest._retry) {
      originalRequest._retry = true;
      localStorage.removeItem('access_token');
      localStorage.removeItem('user');
      if (window.location.pathname !== '/') {
        window.location.href = '/';
      }
    }
    if (!error.response) {
      console.error('Network error:', error);
      return Promise.reject(new Error('Network error. Please check your internet connection.'));
    }
    return Promise.reject(error);
  }
);
```

Листинг 3.7 – Реализация интерсепторов для организации HTTP-запросов

Таким образом, разработанный модуль API является ключевым инфраструктурным элементом клиентской части. Он абстрагирует низкоуровневые детали HTTP-коммуникации, предоставляя разработчикам интерфейса простой и безопасный способ взаимодействия со всеми сервисами бэкенда. Централизованная настройка запросов, обработка токенов и единый подход к вызову серверных методов повышают надежность приложения и упрощают сопровождение кода. Кроме того, такая структура облегчает дальнейшее расширение функциональности, поскольку новые API-методы могут добавляться без изменения общей логики взаимодействия клиента с сервером.

3.2.3 Разработка основного компонента административной панели

Основной компонент панели администратора предназначен для централизованного управления модулями системы. Он реализован на React и предоставляет интерфейс с разделами для работы с пользователями, темами, идеями, сообщениями технической поддержки и элементами краудсорсинга.

Перед отображением административного интерфейса выполняется проверка прав доступа текущего пользователя. Для этого функция `fetchProfile` отправляет асинхронный запрос к API и получает данные профиля. После загрузки данных определяется роль пользователя: сначала используется значение, полученное от сервера, а при его отсутствии роль извлекается из payload JWT-токена, сохраненного в локальном хранилище. На основе результата устанавливается флаг `isAdmin`, который определяет доступность административной панели. Код функции представлен на листинге 3.8.

```
const fetchProfile = async () => {
  try {
    const response = await profileAPI.getProfile();
    const userData = response.data.user;
    setUser(userData);

    let userRole = userData?.role;
    if (!userRole) {
      const token = localStorage.getItem('access_token');
      if (token) {
        const parts = token.split('.');
        if (parts.length === 3) {
          const payload = JSON.parse(atob(parts[1]));
          userRole = payload.role;
        }
      }
    }
    const isAdmin = typeof userRole === 'string'
      ? userRole.toLowerCase() === 'admin'
      : false;
    setIsAdmin(isAdmin);
  } catch (error) {
    console.error('Не удалось загрузить профиль:', error);
    setIsAdmin(false);
  }
};
```

Листинг 3.8 – Функция для определения роли пользователя

Компонент административной панели представляет собой центральный узел управления системой, интегрирующий все ключевые модули. Реализованная логика проверки прав доступа, включающая резервное извлечение роли из JWT-токена, обеспечивает корректное отображение интерфейса в соответствии с ролью пользователя.

3.2.4 Библиотека переиспользуемых UI-компонентов

Для соблюдения принципа DRY [35] и обеспечения единообразия интерфейса выделена отдельная директория `components`, содержащая библиотеку независимых, слабосвязанных компонентов.

Компонент `PrivateRoute.tsx` реализует паттерн защищенного маршрута, ограничивая доступ к определенным страницам только для аутентифицированных пользователей с проверкой наличия токена авторизации.

Компонент `CommentSection` инкапсулирует логику и представление раздела комментариев, включая вывод списка, форму добавления и интерактивные элементы для взаимодействия пользователей с обсуждениями.

Коллекция модальных окон объединяет диалоговые компоненты для различных сценариев взаимодействия с пользователем:

- `LoginModal.tsx`, `RegisterModal.tsx`, `ForgotPasswordModal.tsx` – модальные формы для авторизации, регистрации и восстановления доступа;
- `ConfirmModal.tsx` – универсальный диалог подтверждения действия;
- `BlockUserModal.tsx`, `ContactSupportModal.tsx` – окна для управления пользователями и обращения в службу поддержки;
- `TopicModal.tsx`, `StatisticsModal.tsx` – интерфейсы для управления темами и отображения статистических данных.

3.2.5 Управление стилями

Визуальное единообразие и централизованное управление оформлением обеспечиваются структурой каталога `styles`:

- `globals.css` – определяет глобальные CSS-правила, сброс стилей и базовые типографические настройки для всего приложения.
- `variables.css` – содержит описание CSS-переменных, для управления цветовой палитрой, размерами и другими элементами приложения.
- `animations.css` – централизованное хранилище ключевых кадров и CSS-анимаций, используемых для создания плавных переходов и интерактивности.

3.2.6 Корневые модули и конфигурация

Точка входа `index.tsx` инициализирует процесс рендеринга приложения, осуществляя монтирование корневого компонента `App` в DOM-дерево документа. Файл `App.tsx` выступает в роли структурного ядра приложения, определяя общую компоновку макета и настройки маршрутизации с использованием `React Router`, что позволяет организовать навигацию между страницами без принудительной перезагрузки документа.

Файл `tsconfig.json` задает параметры компиляции `TypeScript`, целевую версию `ECMAScript` и настройки путей к модулям. Файл `package.json` управляет зависимостями проекта, содержит список подключенных библиотек, скрипты сборки и метаданные.

Файлы среды (.env) содержат переменные окружения для гибкой настройки клиентского приложения в различных контекстах развертывания: разработка, тестирование и производство. Это позволяет изменять параметры конфигурации без изменения исходного кода и адаптировать приложение к разным окружениям без пересборки.

3.3. Контейнеризация приложения с использованием Docker

Для обеспечения воспроизводимости развертывания и изолированности компонентов программного комплекса применялась контейнеризация Docker. Каждый ключевой модуль системы – база данных, серверная часть и клиентское приложение – помещен в отдельный контейнер, что позволяет стандартизировать среду выполнения и минимизировать конфликты зависимостей.

Серверная часть, реализованная на фреймворке NestJS, упаковывается с использованием Dockerfile на базе образа node:20-alpine. В процессе сборки устанавливаются зависимости проекта, выполняется компиляция TypeScript-кода и формируется production-образ с запуском скомпилированного приложения через node dist/main. Такой подход обеспечивает изоляцию окружения, упрощает развертывание на серверах и гарантирует идентичность среды на всех этапах жизненного цикла приложения – от разработки до эксплуатации.

Клиентское приложение на React для среды разработки использует отдельный Dockerfile. Особенность его конфигурации – флаг --legacy-peer-deps, обеспечивающий совместимость версий npm-пакетов.

Центральным элементом стратегии развертывания является файл docker-compose.yml, который описывает топологию и взаимодействие всех сервисов системы. В частности, в проекте определены три сервиса:

- postgres – контейнер базы данных PostgreSQL 15, с предварительно настроенной базой ideafLOW_db и механизмом healthcheck для проверки готовности перед запуском зависимых сервисов.

- backend – сервис серверного API, который ожидает полной готовности контейнера postgres перед запуском. В его конфигурации через переменные среды передаются параметры подключения к базе данных, секретный ключ JWT, URL фронтенда и настройки памяти Node.js. На листинге 3.9 представлен фрагмент кода для сервиса серверной части в docker-compose.yml.

- frontend – сервис клиентского приложения с целевым этапом development, зависящий от контейнера backend. Для разработки используются смонтированные volumes, позволяющие синхронизировать изменения исходного кода без пересборки образа и ускорять итеративный процесс разработки.

Полный листинг кода оркестратора для развертывания приложения представлен в приложении К.

Таким образом, применение Docker и Docker Compose в проекте позволило создать согласованную, переносимую среду выполнения. Оркестрация

контейнеров через Compose автоматизирует запуск всей системы одной командой, гарантирует правильный порядок инициализации сервисов и обеспечивает изоляцию компонентов, что упрощает процессы разработки, тестирования и развертывания.

3.4 Выводы по разделу

В ходе разработки было полностью реализовано веб-приложение для краудсорсинга идей, включающее авторизацию, управление контентом, систему реакций, комментарии и административный интерфейс.

Представленная модульная архитектура клиентского приложения характеризуется четким разделением ответственности между компонентами представления, логики и стилями. Применение TypeScript используется для обеспечения статической типизации, что повышает надежность и упрощает долгосрочную поддержку и модификацию программного кода.

Внедрена развитая система аутентификации и авторизации на основе JWT-токенов, дополненная ролевым контролем доступа и механизмом завершения сессии на клиентской стороне. Это гарантирует высокий уровень безопасности и удобство управления сессиями. Интеграция с СУБД через TypeORM обеспечивает целостность данных и простое управление сущностями. EmailModule автоматизирует служебные уведомления и верификацию, снижая ручную нагрузку на администраторов.

Клиентская часть реализована как веб-приложение с использованием React и TypeScript, следуя принципу модульности и компонентного подхода. Такой стек, дополненный React Router DOM для навигации, обеспечивает динамичное и отзывчивое отображение пользовательского интерфейса.

Взаимодействие с API вынесено в специализированные сервисы с использованием Axios. Применение React Hook Form гарантирует надежную обработку пользовательского ввода. Благодаря механизму защищенных маршрутов обеспечивается надежная обработка авторизации и блокировки пользователей на стороне клиента.

Дополнительно реализована система достижений, которая автоматически начисляет бейджи за активность пользователей, стимулируя участие в обсуждениях. Механизм реакций с поддержкой лайков позволяет оценивать идеи и формировать рейтинг контента, а система вложенных комментариев обеспечивает структурированный диалог между участниками.

Таким образом, разработанная архитектура, основанная на четком разделении ответственности между серверным и клиентским компонентами, отличается масштабируемостью и удобством поддержки.

4 Тестирование веб-приложения

Тестирование программного комплекса направлено на проверку функциональной корректности, надежности и соответствия системы исходным требованиям. Оно позволяет выявлять и устранять дефекты до эксплуатации, минимизируя риски для пользователей.

Для всесторонней оценки приложения «IdeaFlow» применяется многоуровневая стратегия, включающая проверку отдельных модулей, их интеграции и работы системы в целом. Особое внимание уделяется ключевым пользовательским сценариям, таким как краудсорсинг идей, модерация и аутентификация.

4.1 Тестирование модуля регистрации и авторизации

Форма регистрации тестируется по следующим сценариях:

- попытка регистрации при незаполненных обязательных полях;
- попытка регистрации с некорректно введенными данными;
- попытка повторной регистрации на существующий почтовый адрес;
- верификация email через код подтверждения;
- обработка неверного или просроченного кода верификации.

На рисунке 4.1 приведен пример поведения формы при вводе некорректных значений в каждое из ее полей.

Регистрация в IdeaFlow

Имя: Имя

Фамилия: Фамилия

Имя обязательно

Фамилия обязательна

Email: blastachanel@gmail.com

Пароль: ****

Пароль должен быть не менее 6 символов

Подтвердите пароль: ****

Зарегистрироваться

Рисунок 4.1. – Результат валидации некорректных значений в полях регистрации

					ДП 04.00.ПЗ		
		ФИО	Подпись	Дата	4 Тестирование веб-приложения		
Разраб.	Телица А.Д.						
Пров.	Сазонова Д.В.						
Н. контр.	Николайчук А.Н.						
Утв.	Блинова Е.А.						
					Лит.	Лист	Листов
					У	1	9
					БГТУ 1-40 05 01, 2026		

В форме регистрации реализована валидация сложности вводимого пароля. Система проверяет пароль на соответствие следующим критериям: минимальная длина 8 символов, наличие хотя бы одной заглавной и одной строчной буквы, а также минимум одной цифры. При несоответствии пароля установленным правилам под полем ввода отображается соответствующее сообщение об ошибке, предотвращая отправку некорректных данных. Результат данной обработки представлен на рисунке 4.2.

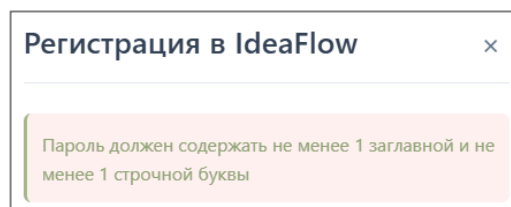


Рисунок 4.2 – Обработка ошибки недостаточной сложности введенного пароля

При попытке регистрации пользователя на уже существующий email, система сообщает об этом. Пример сообщения приведен на рисунке 4.3.

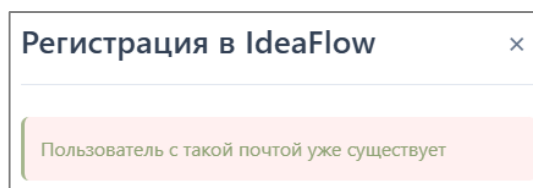


Рисунок 4.3 – Обработка ошибки повторной регистрации на существующий email

Далее рассматривается окно для авторизации пользователей. В этом окне обрабатывается 2 сценария:

- отсутствие данных в полях для заполнения;
- неправильно введенные данные в форму.

На рисунке 4.4 представлен результат обработки вышеописанных сценариев.

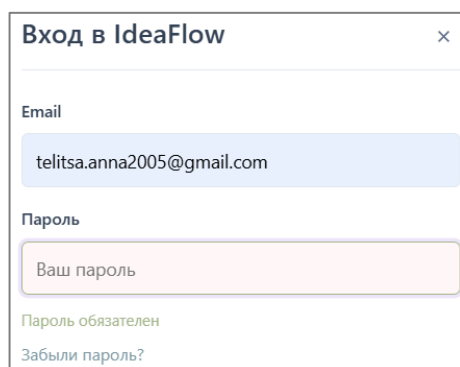


Рисунок 4.4 – Обработка неправильного заполнения формы входа

Таким образом валидируются все данные перед регистрацией пользователей, для избегания некорректных значений.

4.2 Тестирование модуля тем для обсуждения

Пользователь может инициировать создание новой темы для обсуждения, что является одним из ключевых элементов краудсорсинговой платформы. Для этого ему необходимо заполнить форму, представленную на рисунке 4.5. Форма включает обязательные и валидируемые поля: «Название темы» и «Описание». В форме должны быть обязательно заполнены оба поля. В противном случае появляется подсказка, для заполнения данных полей.

Рисунок 4.5 – Форма, в которой пользователь предлагает свою тему

Проведенное тестирование подтвердило корректную работу механизма создания тем пользователями с последующей модерацией. Предложения пользователей проходят стадию предварительного одобрения перед публикацией. Валидация полей формы на стороне клиента и сервера обеспечивает целостность поступающих данных. Механизмы контроля доступа надежно ограничивают видимость тем: пользователи видят только одобренные публичные темы. Обработка статусов темы и связанные с ними ограничения на редактирование работают в соответствии с бизнес-логикой, что в целом обеспечивает управляемый и безопасный процесс краудсорсинга.

4.3 Тестирование модуля идей к темам для обсуждения

В подразделе приведены результаты тестирования модуля идей: создание, изменение, оценка, комментирование, достижения и личная статистика.

4.3.1 Валидация контента при публикации идеи

Для обеспечения качества и информативности пользовательского контента в системе реализована строгая валидация текстового содержимого публикуемых идей. Данное правило действует безусловно, даже в случаях, когда идея

сопровождается графическим вложением, что эффективно ограничивает возможность публикации нерелевантных, недостаточно проработанных или спам-сообщений. Визуальное представление данной функциональности, включая процесс добавления идеи и работу валидации, отображено на рисунке 4.6.

Рисунок 4.6 – Добавление идеи пользователем

Установленное правило ввода минимум 15 символов для описания идеи успешно отсеивает короткие, неинформативные или спам-сообщения, поддерживая общее качество контента на платформе.

4.3.2 Система реакций и геймификации

Механизм оценки идей реализован через систему реакций. Тестирование подтвердило, что каждый пользователь может оставить только одну реакцию на конкретную идею, с возможностью ее отмены при повторном нажатии. Сервис автоматически обновляет счетчики реакций в реальном времени.

Интегрирована система достижений (геймификации), которая поощряет активность пользователей. Реализованы следующие категории достижений:

- Первый автор – за публикацию первой идеи;
- Генератор идей – за публикацию 10 идей;
- Гуру комментариев – за написание 50 комментариев;
- Мастер тем – за создание 5 тем;
- Выбор редакции – за 3 закрепленных администратором идеи;
- Сердце сообщества – за 10 поставленных реакций чужим идеям;
- Вдохновитель – за получение 15 реакций на свои идеи;
- Всезнайка – за комментирование в 5 различных темах.

Достижения отображаются в личном кабинете пользователя и обновляются автоматически при выполнении условий. Работа системы реакций и достижений представлена на рисунке 4.7.

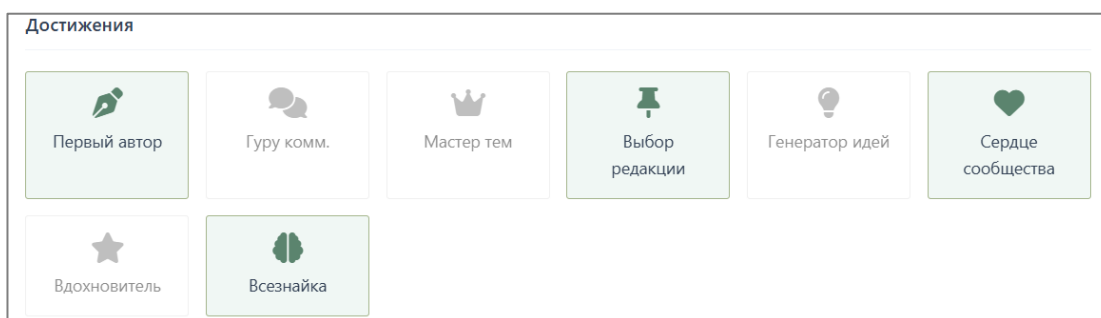


Рисунок 4.7. – Отображение системы реакций и достижений пользователя

Система реакций и достижений работает корректно: один пользователь может поставить только один лайк на идею, а прогресс достижений обновляется автоматически, обеспечивая мотивацию к активности на платформе.

4.3.3 Личный кабинет пользователя

Модуль предоставляет пользователям персонализированное пространство – личный кабинет с аналитическими возможностями. В процессе тестирования проверялась корректность формирования сводной статистики по активности каждого пользователя.

Ключевыми проверяемыми метриками являлись:

- общее количество созданных идей;
- количество оставленных комментариев;
- количество полученных реакций;
- количество поставленных реакций;
- количество закрепленных идей;
- количество тем с оставленными комментариями;
- общий рейтинг пользователя.

Тестирование подтвердило корректность отображения избранных тем, к которым пользователь отмечен повышенный интерес. Обычный пользователь видит только собственную статистику и достижения в рамках своего личного кабинета. Визуальное представление личного кабинета пользователя представлено на рисунке 4.8.

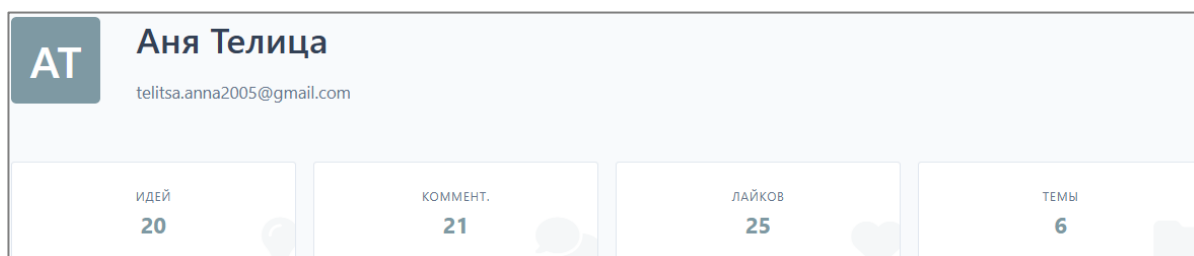


Рисунок 4.8 – Личный кабинет пользователя с достижениями и статистикой

Проведенное тестирование модуля идей подтвердило корректность реализации его ключевых функций. Валидация контента эффективно обеспечивает информативность публикаций. Система реакций работает корректно и гарантирует уникальность оценки от каждого пользователя. Механизмы формирования пользовательской статистики и система достижений функционируют правильно, предоставляя актуальные данные в личном кабинете. В целом модуль создает стабильную и безопасную среду для генерации, обсуждения и оценки идей, полностью соответствующую заданным бизнес-требованиям. Дополнительно было установлено, что обновление статистических показателей происходит своевременно после выполнения пользователем соответствующих действий в системе.

4.4 Тестирование панели администратора

Панель администратора представляет собой привилегированный интерфейс управления системой, реализующий функции контроля пользователей, модерации контента и обработки обращений в поддержку. Тестирование данного модуля направлено на проверку полноты и корректности предоставляемого функционала, строгого соблюдения разграничения доступа на основе ролей, а также устойчивости к некорректным действиям.

4.4.1 Управление пользовательской базой

Проверяется работа просмотра списка пользователей, а также функций блокировки и разблокировки. Особое внимание уделяется корректности указания причины блокировки и ее отображения для заблокированного пользователя. Форма блокировки представлена на рисунке 4.9.

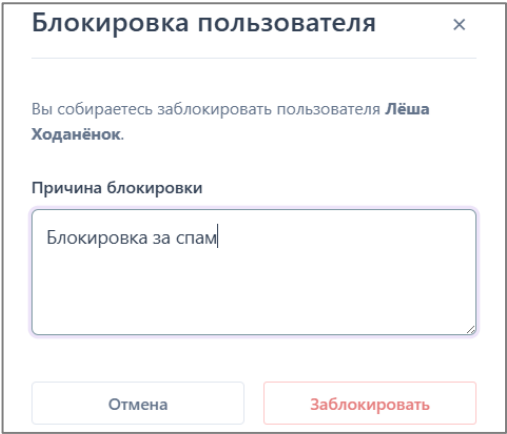


Рисунок 4.9 – Блокировка пользователя администратором

После блокировки аккаунта администратором с указанием причины, данная причина сохраняется в профиле пользователя. При последующей попытке входа система возвращает клиенту ошибку, содержащую данное объяснение блокировки. Визуальное представление данного окна поддержки приведено на рисунке 4.10.

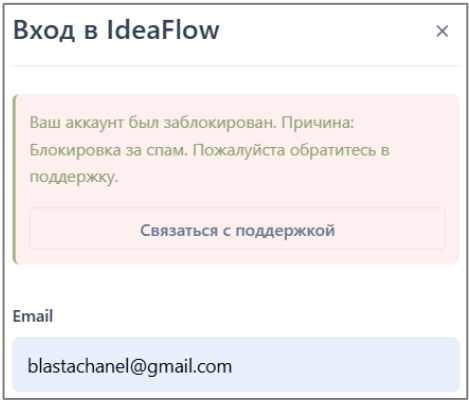


Рисунок 4.10 – Окно службы поддержки для пользователя

Если пользователь отправляет сообщение в службу поддержки, оно отображается у администратора во вкладке «Поддержка». Данное сообщение представлено на рисунке 4.11, где показан интерфейс обработки обращений.

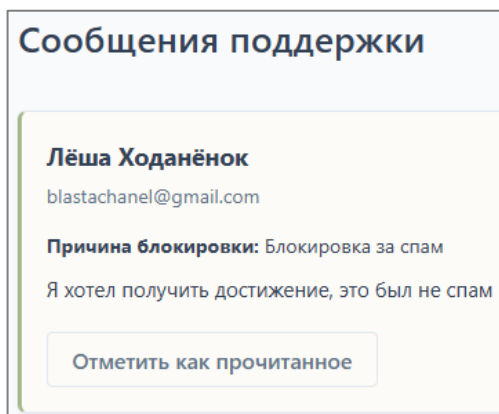


Рисунок 4.11 – Служба поддержки пользователей

После рассмотрения обращения пользователя администратор может разблокировать пользователя в системе аналогично блокировке.

4.4.2 Управление темами для обсуждения

Проверяется полный цикл управления темами администратором. Тестируется корректность создания новых тем с указанием названия, описания, статуса, приватности и дедлайна. Форма администратора приведена на рисунке 4.12.

Рисунок 4.12 – Форма создания темы администратором

В интерфейсе панели администратора корректно реализовано отображение и управление статусами тем для обсуждения. Система визуально разграничивает темы, созданные разными способами: предложенные пользователями отображаются со статусом PENDING и помещаются в очередь на модерацию, а созданные администратором автоматически получают статус

APPROVED и становятся доступными для всех пользователей. Отображение панели администратора при работе с темами показано на рисунке 4.13.

Название	Описание	Статус	Приватность	Дедлайн	Идей	Автор	Действия
Экологичная упаковка для e-commerce: как снизить стоимость и углеродный след	Поиск идей по замене пластика в интернет-торговле на перерабатываемые или...	APPROVED	ПУБЛИЧНЫЙ	01.06.2026	0	System Administrator	Редактировать Удалить
Геймификация школьного обучения: неигровые механики для повышения вовлечённости	Разработка системы достижений, уровней и рейтингов для обычных уроков без превращения всего...	PENDING	ПУБЛИЧНЫЙ		0	Аня Телица	Редактировать Удалить

Рисунок 4.13. – Панель администратора для работы с темами.

Проверяется функционал редактирования существующих тем с сохранением всех связанных идей и комментариев, а также каскадное удаление темы вместе с привязанным контентом. Тестирование подтверждает корректное обновление счетчиков идей в реальном времени после удаления темы.

Вкладка «Главная» предоставляет базовый интерфейс пользователя. Администратор может просматривать идеи, закреплять наиболее значимые в верхней части списка и снимать закрепление. Закрепленные идеи отображаются первыми для всех пользователей, обеспечивая приоритетность контента.

4.4.3 Модерация идей и комментариев

Проверяется административный контроль над пользовательским контентом. Тестируется отображение списка всех идей с указанием темы, количества комментариев и детальный просмотр самих комментариев. Особое внимание уделяется операциям удаления: администратор может удалять отдельные идеи с каскадным удалением комментариев и реакций, а также отдельные комментарии. После удаления контент исчезает из интерфейса пользователей, а счетчики актуализируются в реальном времени. Панель для управления идеями и комментариями представлена на рисунке 4.14.

Название	Автор	Тема	Оценка	Комментарии	Дата создания	Действия
Предлагаю заменить полиэтилен...	Леша Ходанёнок	ЭКОЛОГИЧНАЯ УПАКОВКА для e-commerce: как снизить стоимость и углеродный след	❤️ 4	Скрыть (3)	30 апр. 2026 г., 13:56	Редактировать Удалить
Комментарии к идее:						
Маня Телица					30.04.2026, 15:29:52	Удалить
Альтернативой PLA может служить смесь с крахмалом или целлюлозой, что удешевляет материал и ускоряет биоразложение в обычных условиях.						
Тема: Экологичная упаковка для e-commerce: как снизить стоимость и углеродный след						
Аня Телица					30.04.2026, 15:29:20	Удалить
Увеличение стоимости на 25% на старте может быть компенсировано маркетинговым позиционированием «экологичная доставка» для премиальных брендов.						
Тема: Экологичная упаковка для e-commerce: как снизить стоимость и углеродный след						

Рисунок 4.14 – Взаимодействие администратора с идеями и комментариями

Таким образом была протестирована панель администратора.

4.4.4 Просмотр статистики платформы

Проверяется функционал формирования сводной статистики активности пользователей платформы. Система агрегирует данные и формирует рейтинги в следующих категориях:

- топ создателей тем;
- топ авторов идей;
- топ комментаторов;
- топ получателей реакций.

Тестирование подтвердило корректность агрегации данных и точность подсчета метрик. Статистика обновляется в реальном времени. Панель статистики платформы представлена на рисунке 4.15.

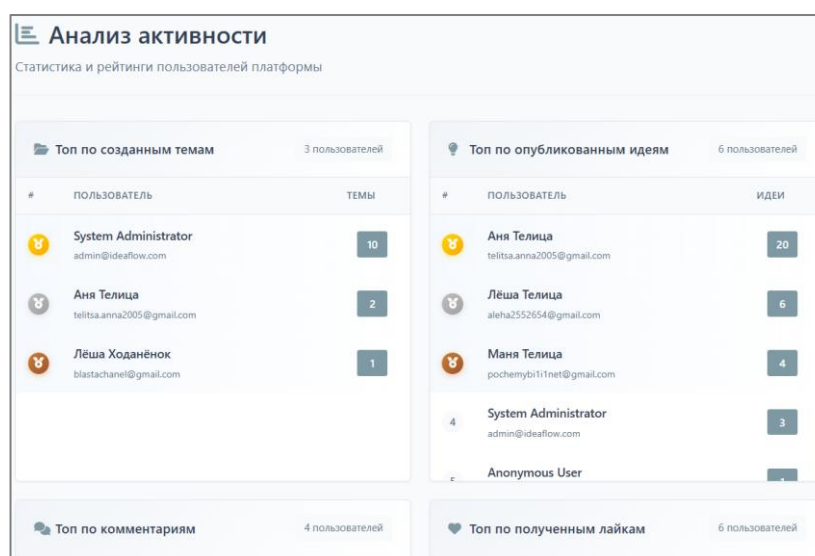


Рисунок 4.15 – Панель статистики платформы с рейтингами пользователей

Статистика формируется автоматически на основе актуальных данных базы и доступна только администраторам, что позволяет оценивать активность сообщества и выявлять наиболее вовлеченных пользователей.

4.5 Выводы по разделу

Проведенное тестирование подтвердило стабильную и безопасную работу ключевых модулей системы «IdeaFlow». Модуль аутентификации обеспечивает регистрацию, вход и контроль доступа с верификацией email. Модуль тем корректно реализует создание, модерацию и управление контентом, включая закрепление важных идей. Модуль идей поддерживает публикацию, реакции, комментарии и достижения. Панель администратора предоставляет инструменты управления пользователями, модерации, обработки обращений и аналитики. Все модули соответствуют требованиям и готовы к эксплуатации.

5 Руководство пользователя

Данный раздел содержит пошаговые инструкции по работе с веб-приложением «IdeaFlow». Материал структурирован по основным сценариям взаимодействия: регистрация и аутентификация, работа с темами для обсуждения, публикация и оценка идей, а также использование панели администратора. Описание ключевых функций сопровождается наглядными скриншотами интерфейса для облегчения освоения платформы.

5.1 Руководство пользователя для роли «Пользователь»

Пользователь может зарегистрироваться в системе. Для этого его необходимо заполнить форму, соблюдая следующие правила:

- пароль должен содержать минимум 1 заглавную букву, 1 строчную букву, 1 цифру и должен быть не менее 8 символов в длину;
- адрес электронной почты должен быть указан корректно;
- пароли должны совпадать;
- поля «Имя» и «Фамилия» должны быть не пустыми.

Форма для регистрации представлена на рисунке 5.1

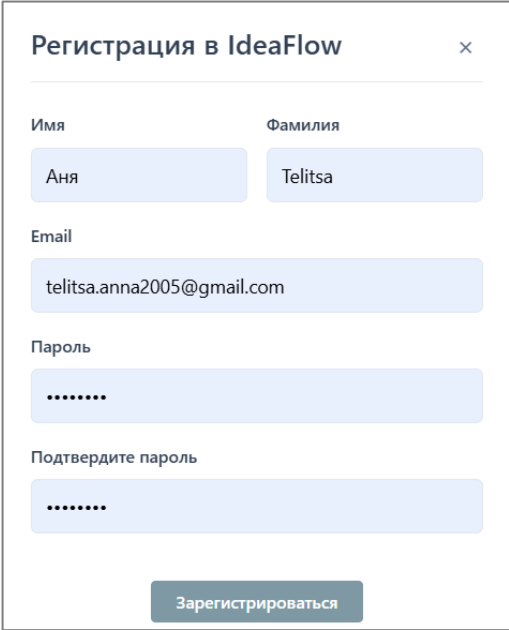


Рисунок 5.1 – Форма для регистрации нового пользователя

После отправки формы на указанный адрес электронной почты автоматически отправляется письмо с шестизначным верификационным кодом.

					ДП 05.00.ПЗ		
		ФИО	Подпись	Дата	5 Руководство пользователя		
Разраб.	Телица А.Д.						
Пров.	Сазонова Д.В.						
Н. контр.	Николайчук А.Н.						
Утв.	Блинова Е.А.						
					Лит.	Лист	Листов
					у	1	9
					БГТУ 1-40 05 01, 2026		

Для активации учетной записи пользователь вводит полученный код на сайте. Без подтверждения email вход в систему невозможен. Форма верификации представлена на рисунке 5.2.

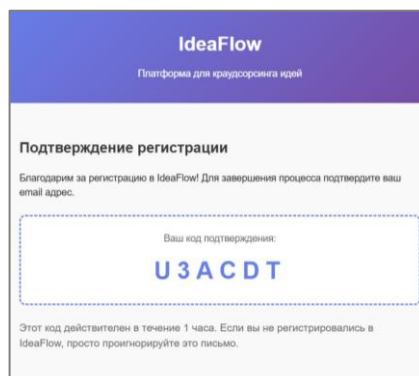


Рисунок 5.2 – Код верификации аккаунта пользователя

После подтверждения регистрации пользователь может войти в систему. Вход осуществляется через форму, представленную на рисунке 5.3.

Рисунок 5.3 – Форма авторизации для пользователей

После успешного входа в свой аккаунт пользователь направляется на главную страницу приложения, которая представлена на рисунке 5.4. Экранная форма главной страницы приложения представлена в приложении Е.

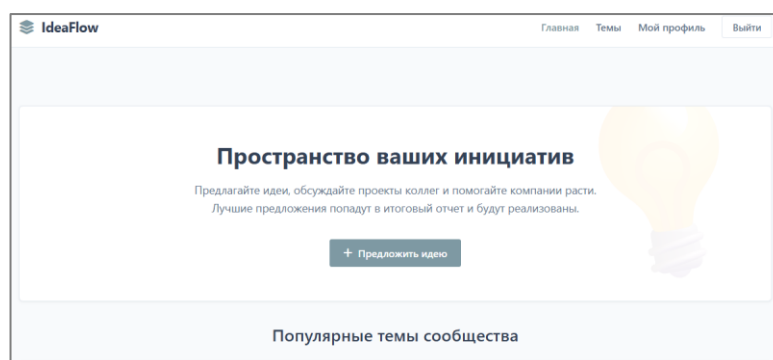


Рисунок 5.4. – Главная страница веб-приложения IdeaFlow для пользователя

На главной странице пользователь может предложить свою тему для обсуждения, выбрав соответствующую кнопку в разделе «Действия». Форма предложения своей идеи представлена на рисунке 5.5.

Рисунок 5.5 – Форма формирования темы пользователем

На главной странице зарегистрированному пользователю отображается список всех доступных для участия тем. Для каждой темы указано название, краткое описание, количество опубликованных идей, автор идеи и срок обсуждения. Пользователь может перейти на страницу любой темы, кликнув по ее заголовку. На странице темы доступны функции для просмотра списка всех идей, опубликованных другими участниками, а также для создания и публикации собственной идеи. Интерфейс одной темы представлен на рисунке 5.6.

Рисунок 5.6 – Карточка выбранной темы

Ниже на странице темы располагается лента опубликованных идей. Для каждой идеи отображается ее описание, автор, дата публикации, количество

реакций и блок комментариев. Визуальное представление ленты идей и структуры комментариев показано на рисунке 5.7.

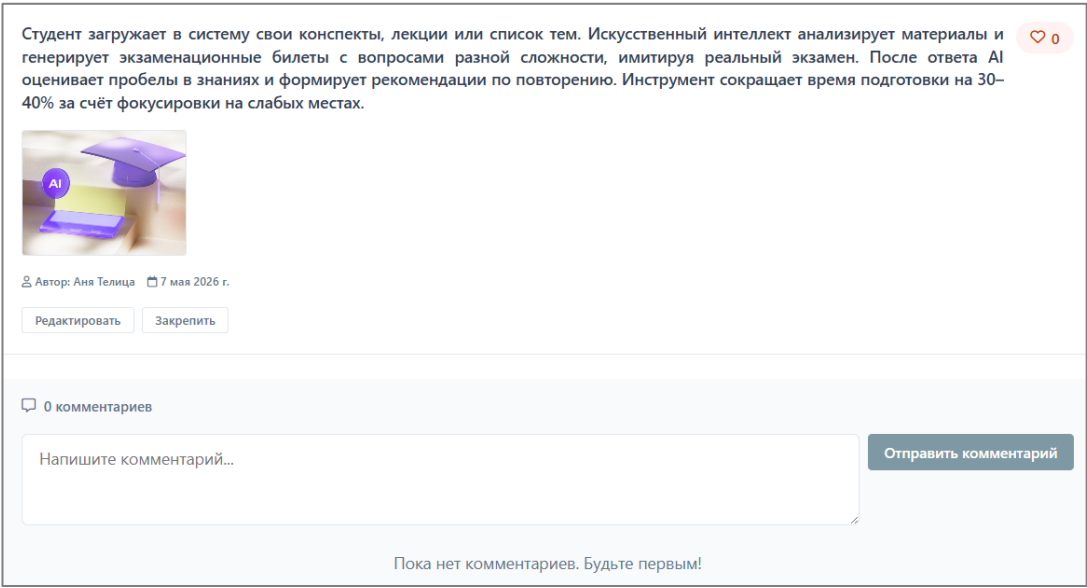


Рисунок 5.7 – Лента идей и комментариев

В данном поле пользователь может оставить свой комментарий к идее, либо ответить на уже существующий комментарий другого пользователя, а также поставить реакцию на идею.

При написании идеи, пользователь может прикрепить до 5 изображений в приложение к своей идее, либо перетащить изображения. Процесс добавления идеи с изображением представлен на рисунке 5.8.

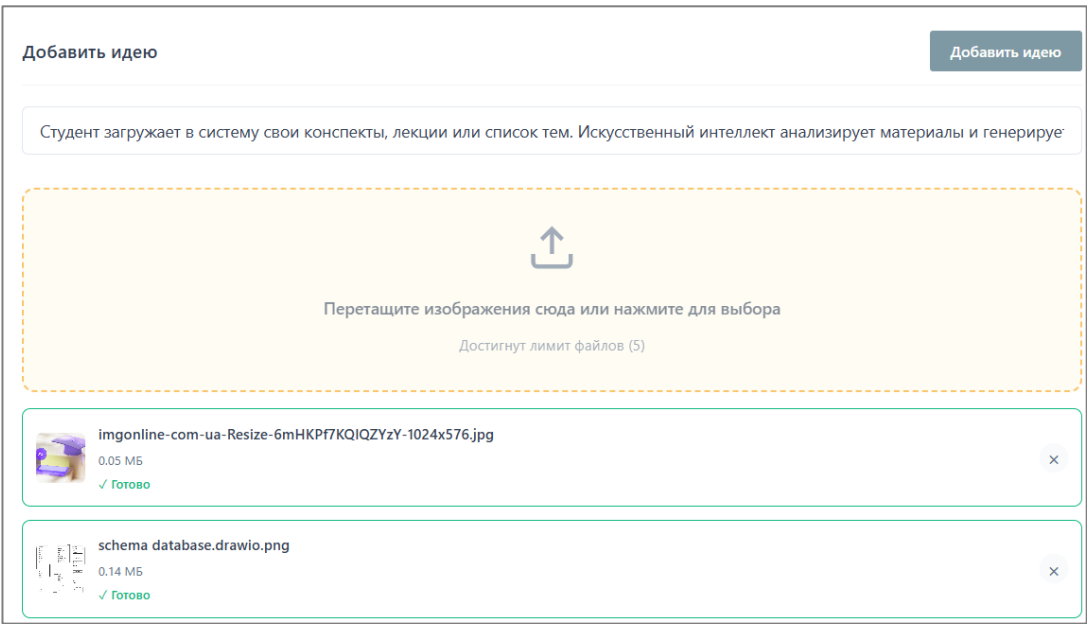


Рисунок 5.8 – Добавление новой идеи пользователем

Также в личном кабинете пользователю доступна его персональная статистика. В аналитику статистики входят все оставленные идеи, комментарии,

а также реакции пользователей. Под статистикой представлена секция с достижениями пользователя, которые он получает за активность в приложении. Окно статистики и достижений пользователя представлено на рисунке 5.9.

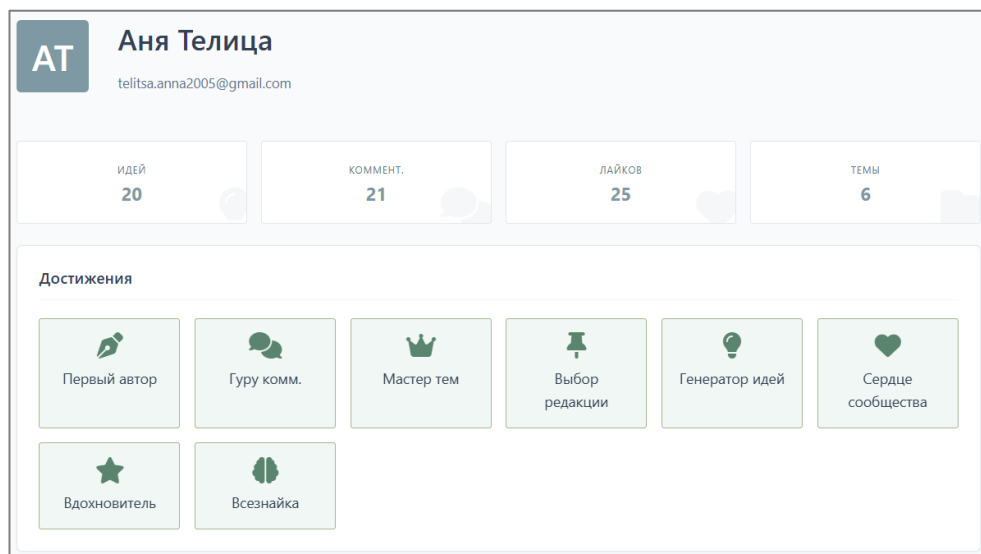


Рисунок 5.9 – Статистика и достижения пользователя

Таким образом, интерфейс зарегистрированного пользователя предоставляет интуитивно понятный набор инструментов для полноценного участия в краудсорсинговом процессе, от поиска интересных тем до публикации и коллективной доработки идей.

5.2 Руководство пользователя для «Администратора»

Регистрация учетной записи администратора происходит автоматически при инициализации системы, поэтому для данной роли сразу доступна авторизация через стандартную форму входа. После аутентификации администратор получает доступ к панели управления, содержащей вкладки: «Пользователи» (управление аккаунтами), «Поддержка» (обработка обращений), «Темы» и «Идеи» (модерация контента), «Анализ» – рейтинги пользователей по различным критериям, «Главная» – базовый интерфейс приложения, идентичный представлению для обычных пользователей, а так же при нажатии на иконку «Администратор», открывается личный кабинет, аналогичный личному кабинету Пользователя, который включает в себя личную статистику, достижения и избранные темы. Такое разделение функций позволяет администратору эффективно переключаться между административными задачами и обычным использованием платформы.

Во вкладке «Пользователи» представлен интерфейс для управления учетными записями. Администратору доступен список всех пользователей с отображением их электронной почты и текущего статуса (активен/заблокирован). Для каждого заблокированного аккаунта указывается причина блокировки, что позволяет отслеживать историю нарушений и принимать обоснованные решения.

Основные инструменты управления включают функцию блокировки пользователя с возможностью указания причины (или без нее) и функцию разблокировки. Администратор не может заблокировать собственную учетную запись или аккаунт другого администратора, что обеспечивает защиту от случайных ошибок и несанкционированных действий. Интерфейс вкладки «Пользователи», а также панель администратора представлены на рисунке 5.10.

Пользователи				
Email	Имя	Статус	Причина блокировки	Действия
admin@ideaflow.com	System Administrator	ACTIVE	—	Администратор
telitsa.anna2005@gmail.com	Аня Телица	ACTIVE	—	Заблокировать
anonymous@ideaflow.com	Anonymous User	ACTIVE	—	Заблокировать
blastachanel@gmail.com	Лёша Ходанёнок	ACTIVE	—	Заблокировать
pochemybi11net@gmail.com	Маня Телица	ACTIVE	—	Заблокировать
aleha2552654@gmail.com	Лёша Телица	BLOCKED	Спам	Разблокировать

Рисунок 5.10 – Интерфейс вкладки управления пользователями

Вкладка «Поддержка» предназначена для обработки обращений пользователей в службу технической поддержки. Администратору доступен список всех сообщений с указанием отправителя, темы обращения, статуса и даты отправки. Функционал позволяет просматривать полное содержание обращения и помечать сообщения как прочитанные для отслеживания состояния обработки. Интерфейс страницы представлен на рисунке 5.11.

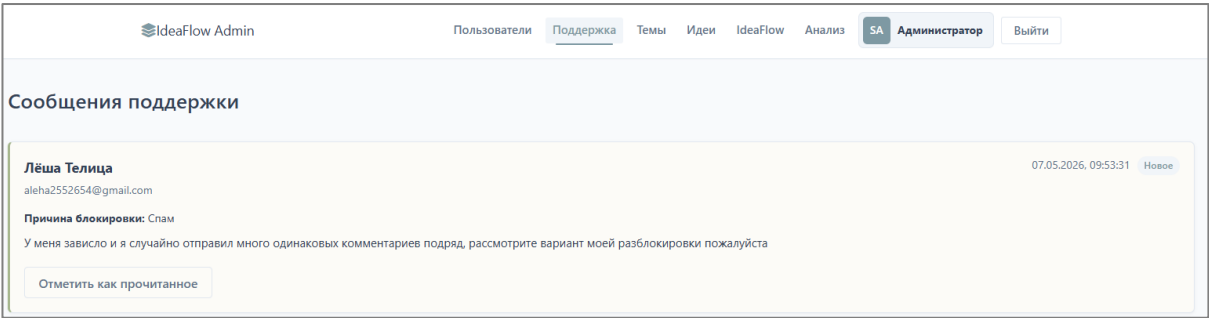


Рисунок 5.11 – Интерфейс службы поддержки

Во вкладке «Темы» администратор получает управление над темами для обсуждения. Здесь отображаются все темы независимо от их статуса, что позволяет администратору иметь полный обзор контента. Для каждой темы доступны инструменты модерации: одобрение, отклонение, редактирование и удаление. Администратор может просматривать предложенные пользователями темы, ожидающие проверки, и публиковать их либо отклонять. При одобрении тема становится доступной для всех пользователей, а при отклонении – скрыта из публичного доступа. Дополнительно администратор может

редактировать содержимое тем, исправлять ошибки формулировок и добавлять дедлайны на темы пользователей. Удаление темы влечет за собой каскадное удаление всех связанных идей и комментариев, что требует аккуратности при выполнении данной операции. Интерфейс вкладки с инструментами модерации представлен на рисунке 5.12.

Темы для обсуждения

+ Создать тему для обсуждения

Название	Описание	Статус	Приватность	Дедлайн	Идей	Автор	Действия
Цифровые инструменты для эффективной подготовки к экзаменам	Какие онлайн-сервисы, приложения и методики помогают студентам быстрее подготовиться к сессии: от систе...	APPROVED	публичный		2	Аня Телица	Редактировать Удалить
Повышение киберграмотности сотрудников через микрообучение и симуляции фишинга	Сбор идей по эффективному обучению персонала основам безопасности без скучных лекций: короткие сцена...	APPROVED	публичный		0	Лёша Ходанёнок	Редактировать Удалить
Экологичная упаковка для e-commerce: как снизить стоимость и углеродный след	Поиск идей по замене пластика в интернет-торговле на перерабатываемые или...	APPROVED	публичный	01.06.2026	1	System Administrator	Редактировать Удалить
Геймификация школьного обучения: неигровые механики для повышения вовлечённости	Разработка системы достижений, уровней и рейтингов для обычных уроков без превращения всего...	PENDING	публичный		0	Аня Телица	Редактировать Удалить

Рисунок 5.12 – Вкладка «Темы»

Также в этом разделе присутствует форма для создания новой темы администратором. В отличие от обычных пользователей, темы, созданные администраторами, сразу появляются в ленте со статусом «одобрено». Форма для создания темы администратором представлена на рисунке 5.13.

Создать тему для обсуждения

Название темы для обсуждения *

Система взаимопроверки студенческих работ без повышения н

Описание *

Обсуждение методов организации анонимного или рецензируемого взаимного оценивания курсовых, эссе и лабораторных работ силами самих студентов: алгоритмы распределения, чек-листы для проверки, балльные системы за качественную рецензию и защита от необъективности.

Приватность

Публичный

Дедлайн

31.05.2026 09:55

Оставьте пустым, если дедлайн не требуется

Отмена

Создать тему для обсуждения

Рисунок 5.13 – Форма создания темы для администратора

Во вкладке «Идеи» сосредоточены инструменты для модерации пользовательского контента. Администратор видит список всех идей, сгруппированных по темам, с информацией об авторе, количестве комментариев и реакций.

Администратор может посмотреть все комментарии к идее, а также имеет возможность удаления конкретных идей или отдельных комментариев в случае нарушения правил платформы. Интерфейс страницы по модерации идей и комментариев представлен на рисунке 5.14.

Название	Автор	Тема	Оценка	Комментарии	Дата создания	Действия
Студент загружает в систему свои...	Аня Телица	ЦИФРОВЫЕ ИНСТРУМЕНТЫ для эффективной подготовки к экзаменам	❤️ 0	Показать (0)	7 мая 2026 г., 09:51	Редактировать Удалить
Студент загружает в систему свои...	Аня Телица	ЦИФРОВЫЕ ИНСТРУМЕНТЫ для эффективной подготовки к экзаменам	❤️ 0	Показать (0)	7 мая 2026 г., 09:51	Редактировать Удалить
Предлагаю заменить полистилен...	Лёша Ходанёнок	ЭКОЛОГИЧНАЯ УПАКОВКА для e-commerce: как снизить стоимость и углеродный след	❤️ 4	Скрыть (3)	30 апр. 2026 г., 13:56	Редактировать Удалить

Комментарии к идее:

Маня Телица

30.04.2026, 15:29:52

Удалить

Альтернативой PLA может служить смесь с крахмалом или целлюлозой, что удешевляет материал и ускоряет биоразложение в обычных условиях.

Тема: Экологичная упаковка для e-commerce: как снизить стоимость и углеродный след

Рисунок 5.14 – Страница модерации идей и комментариев

Вкладка «IdeaFlow» предоставляет администратору доступ к основному интерфейсу приложения, идентичному представлению для обычных пользователей. Ключевое отличие заключается в видимости контента: администратору доступны для просмотра и взаимодействия все темы независимо от их и уровня приватности. Это позволяет администратору полноценно тестировать функционал, участвовать в обсуждениях и оценивать идеи в любом контексте, не покидая панель управления. Кроме того, данная вкладка дает возможность создавать и вести тематические обсуждения исключительно между администраторами, используя закрытые темы. Данный раздел админ панели представлен на рисунке 5.15.

Активные темы обсуждения

Система взаимопроверки студенческих работ без повышения нагрузки на преподавателя

Обсуждение методов организации анонимного или рецензируемого взаимного оценивания курсовых, эссе и лабораторных работ силами самих студентов: алгоритмы распределения, чек-листы для проверки, балльные системы за качественную рецензию и защита от необъективности.

0 идей

До 31.05.2026

Цифровые инструменты для эффективной подготовки к экзаменам

Какие онлайн-сервисы, приложения и методики помогают студентам быстрее готовиться к сессии: от систем карточек (Anki) до AI-помощников и тайм-менеджмента.

2 идеи

Повышение киберграмотности сотрудников через микрообучение и симуляции фишинга

Сбор идей по эффективному обучению персонала основам безопасности без скучных лекций: короткие сценарии, игровые тренажеры, автоматические тестовые фишинговые рассылки.

0 идей

Экологичная упаковка для e-commerce: как снизить стоимость и углеродный след

Поиск идей по замене пластика в интернет-торговле на перерабатываемые или биоразлагаемые материалы без существенного удорожания логистики.

1 идея

До 01.06.2026

Рисунок 5.15 – Страница «IdeaFlow» для администратора

Таким образом, в данном разделе был представлен весь функционал, доступный для администраторов приложения, включая управление пользователями, модерацию контента, обработку обращений в поддержку и аналитические инструменты для оценки активности платформы.

5.3 Выводы по разделу

В данном разделе были рассмотрены подробные пошаговые сценарии работы с платформой «IdeaFlow» для двух ключевых ролей: зарегистрированного пользователя и администратора. Для каждой роли детально описаны доступные функции, навигация по интерфейсу и основные рабочие процессы, что позволяет пользователям быстро освоить приложение и эффективно использовать его возможности.

Руководство для зарегистрированного пользователя охватывает полный цикл действий, доступных в приложении: от регистрации и верификации учетной записи до публикации идей, их оценки и комментирования в рамках доступных тем. Особое внимание уделено системе реакций, работе с избранным и просмотру личной статистики активности, что способствует вовлечению пользователей в краудсорсинговый процесс.

Инструкция для администратора подробно раскрывает возможности панели управления, включая инструменты для модерации контента, управления пользователями, обработки обращений в поддержку и использования специального интерфейса для полномасштабного тестирования системы. Описаны функции блокировки и разблокировки пользователей, одобрения и отклонения тем, закрепления идей и аналитики активности платформы, что обеспечивает администратору полный контроль над состоянием системы и качеством контента.

Важным преимуществом платформы «IdeaFlow» является наличие интуитивно понятного интерфейса, ориентированного на пользователей с различным уровнем цифровой подготовки. Последовательная структура разделов, понятные элементы навигации и наличие вспомогательных функций позволяют сократить время освоения системы и минимизировать вероятность ошибок при выполнении основных операций. Это особенно важно для краудсорсинговых платформ, где удобство взаимодействия напрямую влияет на активность пользователей и качество собираемых предложений.

Таким образом, разработанные инструкции обеспечивают комплексное представление о порядке работы с платформой «IdeaFlow» и могут использоваться как справочный материал при первичном знакомстве с системой. Подробное описание пользовательских и административных возможностей способствует более эффективному внедрению приложения, повышает прозрачность рабочих процессов и создает основу для дальнейшего развития функциональности платформы.

6 Технико-экономическое обоснование проекта

6.1 Общая характеристика разрабатываемого продукта

В результате выполнения дипломного проекта разработано веб-приложение для краудсорсинга идей «IdeaFlow». Программное средство предназначено для сбора, систематизации, обсуждения и отбора предложений пользователей в рамках тематических обсуждений. Система может применяться в организациях, образовательных учреждениях и сообществах, где требуется централизованный инструмент для генерации инициатив, коллективного обсуждения и последующей модерации предложений.

Разработанное приложение предусматривает три категории пользователей: гость, зарегистрированный пользователь и администратор. Гость может зарегистрироваться и выполнить вход в систему. Зарегистрированному пользователю доступны создание и просмотр тем, публикация идей, комментирование, оценка предложений, добавление материалов в избранное и просмотр личной статистики. Администратор обладает расширенными правами: управляет контентом, модерирует темы и идеи, обрабатывает обращения в службу поддержки, блокирует пользователей и анализирует показатели активности.

На основе анализа стратегий монетизации, применяемых продуктами-аналогами, для проекта выбрана модель годовой подписки. В рамках данной модели организация приобретает право использования продукта для своих сотрудников на установленный период.

Техническая реализация проекта выполнена с использованием современного стека веб-разработки: серверная часть реализована на NestJS и TypeScript с применением TypeORM, база данных построена на PostgreSQL, клиентская часть разработана на React.js, а развертывание обеспечивается с использованием Docker и Docker Compose. Такое сочетание технологий обеспечивает масштабируемость, удобство сопровождения, модульность архитектуры и возможность дальнейшего расширения функциональности.

6.2 Исходные данные для проведения расчетов

Разрабатываемое веб-приложение не является принципиально новым видом программного обеспечения, поскольку на рынке существуют платформы для управления идеями, корпоративные порталы предложений и решения для организации обсуждений. При этом «IdeaFlow» обладает собственной спецификой: поддержкой тематических обсуждений, пользовательских идей, комментариев, реакций, избранного, достижений, личной статистики, панели администратора.

					ДП 06.00.ПЗ						
		ФИО	Подпись	Дата							
Разраб.	Телица А.Д.				6 Технико-экономическое обоснование проекта	Лит.		Лист		Листов	
Пров.	Сазонова Д.В.					У		1		11	
Консульт.	Соболевский А.С.					БГТУ 1-40 05 01, 2026					
Н. контр.	Николайчук А.Н.										
Утв.	Блинова Е.А.										

По уровню сложности программное средство можно отнести ко второй группе сложности, так как оно включает клиент-серверную архитектуру, базу данных, авторизацию, разграничение ролей, административную панель, модерацию пользовательского контента и набор интерактивных пользовательских функций. Из перечня показателей, предусмотренных методикой, для Расчета выбраны те исходные данные, которые соответствуют способу оценки данного проекта: Расчету затрат на разработку веб-приложения, сопровождение, плановую прибыль и отпускную цену. Показатели, связанные с тарифной ставкой первого разряда, тарифным коэффициентом и стоимостью машино-часа, не используются, так как основная заработная плата определяется на основании средних месячных заработных плат ИТ-специалистов. Основные исходные данные, использованные для Расчета экономических показателей, приведены в таблице 6.1.

Таблица 6.1 – Исходные данные для расчета

Наименование показателя	Условные обозначения	Норматив
Численность разработчиков, чел	Ч_p	1
Фонд рабочего времени в месяце, ч	ФРВ	21
Стоимость одного машино-часа, руб	$\text{С}_{\text{м.ч.}}$	0,05
Норматив дополнительной заработной платы, %	$\text{Н}_{\text{доп.з.п.}}$	15
Ставка отчислений в Фонд социальной защиты населения, %	$\text{Н}_{\text{ф.с.з.н.}}$	34
Ставка отчислений по обязательному страхованию от несчастных случаев на производстве и профессиональных заболеваний, %	$\text{Н}_{\text{б.г.с.}}$	0,6
Норматив прочих прямых затрат, %	$\text{Н}_{\text{п.з.}}$	18
Норматив общепроизводственных и общехозяйственных расходов, %	$\text{Н}_{\text{обп.обх.}}$	30
Норматив расходов на сопровождение и адаптацию, %	$\text{Н}_{\text{р.с.а.}}$	10
Ставка НДС, %	$\text{Н}_{\text{ндс.}}$	20

6.3 Методика обоснования цены

В условиях рыночной экономики веб-приложение рассматривается как Законченный программный продукт может быть использован заказчиком для автоматизации внутренних процессов или представлен пользователям как самостоятельный сервис. Экономическая эффективность разработки определяется сопоставлением затрат на создание программного средства с возможной ценой его реализации и выгодой от применения. Для пользователя применение «IdeaFlow» позволяет сократить время на сбор и обработку предложений, упростить коммуникацию между участниками, повысить прозрачность модерации и обеспечить централизованное хранение идей и обсуждений.

Оценка стоимости программного продукта включает определение следующих видов затрат:

- основная заработная плата исполнителей;
- дополнительная заработная плата;
- отчисления на социальные цели;

- расходы на материалы;
- расходы на специальное оборудование и платные услуги;
- прочие прямые затраты;
- общепроизводственные и общехозяйственные расходы;
- расходы на сопровождение и адаптацию.

На основании перечисленных затрат рассчитываются себестоимость разработки, полная себестоимость программного средства, плановая прибыль, цена разработчика и отпускная цена с учетом НДС.

6.4 Расчет затрат рабочего времени на разработку программного средства

Затраты рабочего времени определяются по основным этапам проектирования, реализации, тестирования и сопровождения разработки. В расчете учитываются специалисты, привлекаемые к созданию программного средства, а также ориентировочная продолжительность выполнения соответствующих работ. Информация о затратах рабочего времени на разработку программного средства приведена в таблице 6.2.

Таблица 6.2 – Затраты рабочего времени на разработку программного средства

Содержание работ	Исполнитель	Количество исполнителей	Затраты рабочего времени, дней
Согласование технического задания с заказчиком	проджект-менеджер	1	5
Анализ требований и предметной области	бизнес-аналитик	1	12
Проектирование пользовательского интерфейса	UX/UI дизайнер	1	9
Разработка серверной части и настройка базы данных	Backend-разработчик	1	34
Разработка клиентской части	Frontend-разработчик	1	22
Контейнеризация и развертывание	DevOps-инженер	1	5
Функциональное тестирование приложения	тестировщик	1	14
Написание руководства пользователя	бизнес-аналитик	1	2
Координация и управление проектом	Проджект-менеджер	1	3
Всего			106

6.5 Расчет основной заработной платы

Для расчета основной заработной платы исполнителей, участвующих в разработке веб-приложения IdeaFlow, учитываются трудоемкость выполнения работ, количество специалистов по каждой роли и средние месячные заработные платы на рынке в 2026 году. В состав проекта входят анализ требований,

проектирование интерфейса и базы данных, разработка серверной и клиентской частей, контейнеризация, тестирование и координация работ. Размер заработной платы выбран с учетом сложности выполняемых задач и рыночного уровня Junior-специалистов. В расчетах принято, что среднемесячный фонд рабочего времени составляет 21 рабочий день.

Основная заработная плата будет рассчитываться по формуле

$$C_{оzi} = T_{рази} \cdot C_{зпи} \cdot K_{рази}, \quad (6.1)$$

где $C_{оzi}$ – основная заработная плата, руб.;

$T_{рази}$ – время раработки, месяцев;

$C_{зпи}$ – средняя заработная плата, руб.;

$K_{рази}$ – количество разработчиков, человек.

Время разработки в месяцах определяется по формуле

$$T_{рази} = T_{дни} / \text{ФРВ}, \quad (6.2)$$

где $T_{дни}$ – трудоемкость работ i-го исполнителя, дней;

ФРВ – фонд рабочего времени в месяце, дней.

Данные исследования, расчет основной заработной платы и времени разработки приведены в таблице 6.3.

Таблица 6.3 – Расчет основной заработной платы.

Исполнитель	Средняя заработная плата, руб./мес.	Затраты времени, дней	Затраты времени, мес.	Основная заработная плата, руб.
Проджект-менеджер	2198,35	8	0,3810	837,47
Бизнес-аналитик	1946,37	14	0,5714	1112,21
UX/UI-дизайнер	2078,92	9	0,4286	890,97
Backend-разработчик (NestJS/TypeScript)	2496,73	26	1,3333	3328,97
Разработчик баз данных PostgreSQL	2327,19	8	0,3810	886,55
Frontend-разработчик React	2264,58	22	1,0476	2372,42
DevOps-инженер	2391,46	5	0,2381	569,40
Тестировщик	1818,74	14	0,6667	1212,49
Итого		106		11 210,47

Исходя из приведенных расчетов, основная заработная плата исполнителей, привлекаемых к разработке веб-приложения IdeaFlow, составила 11210,47 руб. В дальнейшем для других расчетов используется основная заработная плата, рассчитанная по указанной выше методике.

6.6 Расчет дополнительной заработной платы

Дополнительная заработная плата включает выплаты, предусмотренные законодательством о труде Республики Беларусь, и определяется по нормативу в процентах к основной заработной плате и формуле

$$C_{\text{дз}} = \frac{C_{\text{оз}} \cdot N_{\text{дз}}}{100}, \quad (6.3)$$

где $C_{\text{дз}}$ – дополнительная заработная плата, руб.;

$C_{\text{оз}}$ – основная заработная плата, руб.;

$N_{\text{дз}}$ – норматив дополнительной заработной платы, %.

$$C_{\text{дз}} = 11210,47 \cdot 15 / 100 = 1681,57 \text{ руб.}$$

Таким образом, дополнительная заработная плата составила 1681,57 руб.

6.7 Расчет отчислений на социальные цели

Отчисления в Фонд социальной защиты населения и взносы по обязательному страхованию от несчастных случаев на производстве и профессиональных заболеваний относятся к обязательным начислениям на заработную плату исполнителей. Они рассчитываются от суммы основной и дополнительной заработной платы и учитываются при определении общей себестоимости разработки программного средства.

Отчисления в Фонд социальной защиты населения рассчитываются по формуле

$$C_{\text{фсзн}} = \frac{(C_{\text{оз}} + C_{\text{дз}}) \cdot N_{\text{фсзн}}}{100}, \quad (6.4)$$

где $C_{\text{оз}}$ – основная заработная плата, руб.;

$C_{\text{дз}}$ – дополнительная заработная плата, руб.;

$N_{\text{фсзн}}$ – норматив отчислений в Фонд социальной защиты, %.

Отчисления по обязательному страхованию рассчитываются по формуле

$$C_{\text{бгс}} = \frac{(C_{\text{оз}} + C_{\text{дз}}) \cdot N_{\text{бгс}}}{100}, \quad (6.5)$$

где $N_{\text{бгс}}$ – норматив отчислений в БРУСП «Белгосстрах», %.

$$C_{\text{фсзн}} = (11210,47 + 1681,57) \cdot 34 / 100 = 4383,49 \text{ руб.}$$

$$C_{\text{бгс}} = (11210,47 + 1681,57) \cdot 0,6 / 100 = 77,35 \text{ руб.}$$

Общие отчисления на социальные цели составили 4460,64 руб., в том числе 4383,29 руб. в Фонд социальной защиты населения и 77,35 руб. в «Белгосстрах».

6.8 Расчет расходов на материалы

Сумма расходов на материалы определяется на основании фактических затрат, связанных с подготовкой документации и организацией разработки программного средства и представлена в таблице 6.4.

Таблица 6.4 – Материальные затраты на разработку ПС

Вид материальных затрат	Фактические затраты, руб.
Бумага, 0,75 пачки	9,86
Картридж Canon, 0,5 шт.	15,00
Всего	24,86

Сумма расходов на материалы составила 24,86 руб.

6.9 Расчет расходов на специальное оборудование и платные услуги

Для разработки веб-приложения «IdeaFlow» использовались доступные программные средства и технологии с открытым доступом: NestJS, TypeScript, TypeORM, PostgreSQL, React.js, Docker и Docker Compose. Приобретение специального оборудования и платных услуг для реализации проекта не осуществлялось. Расходы на специальное оборудование и платные услуги составили 0,00 руб.

6.10 Расчет прочих прямых затрат

Прочие прямые затраты включают расходы, связанные с поиском и подготовкой справочной, нормативной и технической информации, организацией процесса разработки и обеспечением выполнения проектных работ. Они рассчитываются по нормативу к основной заработной плате по формуле

$$C_{\text{пз}} = \frac{C_{\text{оз}} \cdot H_{\text{пз}}}{100} \quad (6.5)$$

$$C_{\text{пз}} = 11210,47 \cdot 18 / 100 = 2017,88 \text{ руб.}$$

Сумма прочих прямых затрат составила 2017,88 руб.

6.11 Расчет общепроизводственных и общехозяйственных расходов

Общепроизводственные и общехозяйственные расходы отражают затраты организационного характера, связанные с обеспечением условий разработки: использованием рабочего места, электроэнергии, связи, вычислитель-

ной техники, администрированием проекта и эксплуатацией общей инфраструктуры. Сумма расходов рассчитывается по формуле

$$C_{\text{обп, обх}} = \frac{C_{\text{оз}} \cdot H_{\text{обп, обх}}}{100}, \quad (6.6)$$

где $C_{\text{обп, обх}}$ – общепроизводственные и общехозяйственные расходы, руб.;

$H_{\text{обп, обх}}$ – норматив общепроизводственных и общехозяйственных расходов, %.

$$C_{\text{обп, обх}} = 11210,47 \cdot 30 / 100 = 3\,363,14 \text{ руб.}$$

Размер общепроизводственных и общехозяйственных расходов составил 3 363,14 руб.

6.12 Расчет суммы расходов на разработку программного средства

Сумма расходов на разработку программного средства определяется как сумма основной и дополнительной заработной платы исполнителей, отчислений на социальные цели, расходов на материалы, расходов на специальное оборудование и платные услуги, прочих прямых затрат и общепроизводственных и общехозяйственных расходов рассчитывается по формуле

$$C_p = C_{\text{оз}} + C_{\text{дз}} + C_{\text{фсзн}} + C_{\text{бгс}} + C_{\text{м}} + C_{\text{сопу}} + C_{\text{пз}} + C_{\text{обп, обх}}. \quad (6.7)$$

С учетом рассчитанных ранее значений сумма расходов составляет

$$C_p = 11210,47 + 1681,57 + 4383,29 + 77,35 + 24,86 + 0,00 + 2017,88 + 3363,14 = 22\,758,56 \text{ руб.}$$

Сумма расходов на разработку программного средства составила 22 758,56 руб.

6.13 Расчет расходов на сопровождение и адаптацию

Расходы на сопровождение и адаптацию учитывают затраты, связанные с первоначальной настройкой программного средства, исправлением выявленных замечаний, адаптацией к условиям эксплуатации и консультационной поддержкой пользователей.

Расчет выполняется по формуле

$$C_{\text{р.с.а}} = \frac{C_p \cdot H_{\text{р.с.а}}}{100}, \quad (6.8)$$

где $C_{\text{р.с.а}}$ – расходы на сопровождение и адаптацию, руб.;

$H_{\text{р.с.а}}$ – норматив расходов на сопровождение и адаптацию, %.

После подстановки вычисленных показателей получается значение:

$$C_{pca} = 22\,758,56 \cdot 10 / 100 = 2275,86 \text{ руб.}$$

Расходы на сопровождение и адаптацию составили 2275,86 руб.

6.14 Расчет полной себестоимости программного средства

Полная себестоимость программного средства определяется как сумма расходов на разработку и расходов на сопровождение и адаптацию по формуле

$$C_{\pi} = C_p + C_{p.c.a.} \quad (6.9)$$

$$C_{\pi} = 22758,56 + 2275,86 = 25034,42 \text{ руб.}$$

Полная себестоимость веб-приложения «IdeaFlow» составила 25034,42 руб.

6.15 Определение цены и оценка эффективности

Разрабатываемое веб-приложение «IdeaFlow» ориентировано на использование в корпоративной среде. В отличие от массовых пользовательских приложений, реализация данного программного средства предполагает продажу годовой корпоративной подписки организации. Приобретая подписку, компания получает возможность предоставить сотрудникам доступ к функциональности приложения: созданию и обсуждению идей, комментированию, использованию реакций, избранного, достижений, личной статистики, административной панели, службы поддержки и механизмов модерации.

Для оценки рыночной цены годовой корпоративной подписки были рассмотрены продукты-аналоги, которые могут использоваться компаниями для организации коммуникации, обсуждения инициатив и управления идеями:

1. Пряники – российская платформа для корпоративной культуры, позволяющая сотрудникам обмениваться идеями, участвовать в опросах и награждать коллег за достижения.

2. Brightidea – международная краудсорсинговая платформа для управления инновациями, предназначенная для сбора, оценки и реализации идей от сотрудников и партнеров компании.

3. IdeaScale – специализированная платформа для сбора, обсуждения и оценки идей, ориентированная на управление инновациями и инициативами.

Для сравнения выбраны показатели качества, значимые для корпоративной платформы управления идеями: функциональность работы с идеями, удобство использования, модерация и администрирование, аналитика и статистика, простота внедрения.

Выбор показателей обусловлен спецификой корпоративного применения платформы. Функциональность определяет возможности создания, редактирования и оценки предложений; удобство использования влияет на скорость внед-

рения и вовлеченность сотрудников; модерация обеспечивает контроль контента; аналитика позволяет оценивать эффективность инициатив; простота внедрения характеризует скорость интеграции в бизнес-процессы организации.

Балльная оценка выполняется экспертным методом, а весовые коэффициенты определяются с учетом значимости критериев для корпоративного заказчика. Сумма весовых коэффициентов составляет 1,00. Расчет показателей качества приведен в таблице 6.5.

Таблица 6.5 – Сравнительная оценка качества программных средств

Показатель качества	Весовой коэффициент	IdeaFlow	Пряники	BrightIdea	IdeaScale
Функциональность работы с идеями	0,3	8	7	9	9
Удобство использования	0,2	8	8	7	7
Модерация и администрирование	0,2	8	7	8	8
Аналитика и статистика	0,15	7	6	9	9
Простота внедрения	0,15	8	8	6	6
Всего	1	7,85	7,25	7,9	7,8

Интегральный показатель качества веб-приложения «IdeaFlow» составил 7,85, что сопоставимо с уровнем ведущих аналогов: Brightidea – 7,90, IdeaScale – 7,80, «Пряники» – 7,25. Незначительное отклонение от максимального значения показывает, что разрабатываемое решение обладает близким уровнем функциональности при сохранении собственной специфики. Полученный результат подтверждает конкурентоспособность программного средства в области корпоративного сбора, обсуждения и модерации идей, а также его пригодность для использования организациями.

Расчет прогнозного количества установок программного средства K_1 при монетизации методом подписок, рассчитывается по формуле

$$K_1 = (K_0 / T_0 \cdot \text{ИР}) / \text{ИК} \quad (6.10)$$

где K_0 – количество установок ПС конкурента;

T_0 – количество лет существования приложения;

ИР – показатель рассматриваемого программного продукта;

ИК – показатель программного продукта конкурента.

$$K_1 = (22 / 4 \cdot 7,85) / 7,25 = 5,96 \text{ (подписок в год),}$$

$$K_2 = (18 / 3 \cdot 7,85) / 7,9 = 5,83 \text{ (подписок в год),}$$

$$K_3 = (18 / 3 \cdot 7,85) / 7,8 = 6,04 \text{ (установок в год),}$$

$$K = (5,96 + 5,83 + 6,04) / 3 = 5,99 \text{ (установок в год).}$$

Таким образом, на основе анализа рыночных данных продуктов-аналогов прогнозируется реализация 6 годовых корпоративных подписок в первый год эксплуатации. Расчетная цена годовой корпоративной подписки определяется путем сопоставления стоимости аналогичных решений и корректировки с учетом показателей качества разрабатываемого продукта по формуле

$$\Pi_p = (\Pi_0 \cdot \text{ИР})/\text{ИК}, \quad (6.11)$$

где Π_0 – цена программного продукта конкурента,

ИР – показатель рассматриваемого программного продукта,

ИК – показатель программного продукта конкурента.

Для расчета приняты ориентировочные годовые стоимости корпоративного использования продуктов-аналогов, применяемых для управления идеями и организации коллективной работы с предложениями. Данные значения используются для сопоставления рыночного уровня цен и определения обоснованной стоимости подписки на веб-приложение IdeaFlow. Расчет цены подписки представлен в таблице 6.6.

Таблица 6.6 – Средние цены подписок на продукты-аналоги

Продукт-аналог	Годовая стоимость продукта-аналога, руб.	Интегральный показатель аналога	Расчетная цена подписки IdeaFlow, руб.
Пряники	5100	7,25	5522,07
Brightidea	10200	7,9	10135,44
IdeaScale	10800	7,8	10869,23
Среднее значение			8842,25

Средняя расчетная цена годовой корпоративной подписки составляет 8842,25 руб. Для дальнейших расчетов принимается отпускная цена с учетом НДС:

$$\Pi_{\text{подп}} = 8800 \text{ руб. в год}$$

Цена подписки без НДС определяется по формуле

$$\Pi_{\text{без НДС}} = \frac{\Pi_{\text{подп}}}{1,2}, \quad (6.12)$$

$$\Pi_{\text{без НДС}} = \frac{8800}{1,2} = 7333,33 \text{ руб.}$$

Выручка разработчика от одной годовой корпоративной подписки без учета НДС составляет 7333,33 руб.

Прогнозируемое количество корпоративных клиентов в первый год – 6 подписок. Годовая выручка без учета НДС рассчитывается по формуле

$$V_{\text{год}} = \Pi_{\text{без НДС}} \cdot K_k, \quad (6.13)$$

где $V_{\text{год}}$ – годовая выручка без учета НДС, руб.,
 K_k – количество корпоративных клиентов за год,
 $ИК$ – показатель программного продукта конкурента.

$$V_{\text{год}} = 7333,33 \cdot 6 = 43999,98 \text{ руб.}$$

Годовые денежные поступления с учетом НДС:

$$П_{\text{год}} = 8800,00 \cdot 6 = 52800,00 \text{ руб.}$$

Количество корпоративных клиентов, необходимое для окупаемости разработки рассчитывается по формуле

$$K_{\text{окуп}} = \frac{C_{\text{п}}}{C_{\text{без НДС}}}, \quad (6.14)$$

где $K_{\text{окуп}}$ – количество корпоративных клиентов, необходимое для окупаемости,
 $C_{\text{п}}$ – полная себестоимость программного средства, руб.

$$K_{\text{окуп}} = \frac{25034,42}{7333,33} = 3,42,$$

Для окупаемости проекта необходимо реализовать не менее 4 годовых корпоративных подписок.

Срок окупаемости проекта рассчитывается по формуле

$$T_{\text{ок}} = \frac{C_{\text{п}}}{V_{\text{год}}}, \quad (6.15)$$

где $T_{\text{ок}}$ – срок окупаемости, лет.

$$T_{\text{ок}} = 25034,42 / 43999,98 = 0,57 \text{ года.}$$

При продаже 6 годовых корпоративных подписок срок окупаемости составит около 0,57 года. Проект экономически целесообразен и обладает потенциалом коммерческого использования в корпоративной среде.

6.16 Вывод по разделу

Итоговые показатели позволяют оценить экономическую эффективность разработки, определить объем необходимых затрат и сопоставить их с ожидаемыми доходами от коммерческого использования программного средства. В расчетах учитывались трудозатраты, заработная плата, обязательные отчисления, прямые и косвенные расходы, а также затраты на сопровождение

и адаптацию продукта. Это обеспечивает комплексную оценку стоимости проекта и позволяет определить минимальное количество клиентов, необходимое для его окупаемости. В таблице 6.7 представлены результаты расчетов для основных показателей данной главы в краткой форме.

Таблица 6.7 – Результаты расчетов

Наименование показателя	Значение
Время разработки, мес.	5,05
Количество программистов, чел.	1
Основная заработная плата, руб.	11210,47
Дополнительная заработная плата, руб	1681,57
Отчисления в Фонд социальной защиты населения и по обязательному страхованию от несчастных случаев на производстве и профессиональных заболеваний, руб	4460,64
Расходы на материалы, оплату машинного времени, руб	24,86
Прочие прямые затраты, руб	2017,88
Общепроизводственные и общехозяйственные расходы, руб.	3363,14
Себестоимость разработки программного средства, руб.	22758,56
Расходы на сопровождение и адаптацию, руб.	2275,86
Полная себестоимость, руб.	25034,42
Цена годовой корпоративной подписки с НДС, руб.	8800
Количество клиентов для окупаемости	4
Срок окупаемости, лет	33

Таблица расчета затрат на разработку представлена в приложении Ж.

По результатам расчетов установлено, что разработка веб-приложения «IdeaFlow» является экономически целесообразной. Общая трудоемкость проекта составила 106 рабочих дней, или 5,05 месяца, а полная себестоимость разработки с учетом сопровождения и адаптации – 25 034,42 руб.

Для коммерческого использования выбрана модель годовой корпоративной подписки стоимостью 8 800 руб. с НДС, или 7 333,33 руб. без НДС. При реализации 6 подписок в первый год выручка без НДС составит 43 999,98 руб. Для окупаемости проекта необходимо продать не менее 4 подписок, а плановый срок окупаемости составит 0,57 года, то есть около 6–7 месяцев.

Сравнение с аналогами «Пряники», Brightidea и IdeaScale подтвердило конкурентоспособность «IdeaFlow»: интегральный показатель качества составил 7,85. Таким образом, проект имеет обоснованную себестоимость, понятную модель монетизации, короткий срок окупаемости и перспективы коммерческого использования.

Заключение

В ходе выполнения данной работы была спроектирована, реализована и протестирована веб-платформа для краудсорсинга идей «IdeaFlow». Система предоставляет полноценную экосистему для генерации, обсуждения и коллективной оценки пользовательских предложений в рамках тематических обсуждений.

Техническая реализация продемонстрировала эффективность выбранного стека технологий: бэкенд на NestJS с TypeORM обеспечил создание масштабируемой, модульной архитектуры с четким разделением ответственности; фронтенд на React позволил построить интуитивно понятный и отзывчивый интерфейс; контейнеризация с помощью Docker и Docker Compose гарантировала воспроизводимость и изолированность среды развертывания. Ключевые бизнес-процессы, такие как дуальная система создания тем, трехуровневая модерация контента и ролевая модель доступа, были успешно инкапсулированы в соответствующих модулях.

Результаты комплексного тестирования подтвердили стабильность, безопасность и корректность работы всех модулей системы. Функциональность охватывает полный жизненный цикл взаимодействия пользователя с платформой – от регистрации и верификации до публикации идей и участия в обсуждениях, а также предоставляет администратору достаточный набор инструментов для управления системой и контентом.

Таким образом, поставленные цели проекта достигнуты: разработано законченное, работоспособное веб-приложение, которое решает задачу организации краудсорсинговой деятельности. Система обладает продуманной архитектурой, отвечает современным требованиям к безопасности и пользовательскому опыту и готова к опытной эксплуатации. Дальнейшее развитие проекта может быть направлено на расширение функционала, интеграцию с внешними сервисами и оптимизацию производительности под высокие нагрузки.

					ДП 00.00.ПЗ				
		ФИО	Подпись	Дата					
Разраб.	Телица А.Д.				Заключение	Лит.	Лист	Листов	
Пров.	Сазонова Д.В.					у	1	1	
						БГТУ 1-40 05 01, 2026			
Н. контр.	Николайчук А.Н.								
Утв.	Блинова Е.А.								

Список использованных источников

- 1 TypeScript : [сайт]. – 2026. – URL: <https://www.typescriptlang.org/> (дата обращения: 29.01.2026).
- 2 NestJS Documentation : [сайт]. – 2026. – URL: <https://docs.nestjs.com/> (дата обращения: 29.01.2026).
- 3 React : [сайт]. – 2026. – URL: <https://react.dev/> (дата обращения: 29.01.2026).
- 4 JWT : [сайт]. – 2026. – URL: <https://jwt.io/introduction> (дата обращения: 29.01.2026).
- 5 Docker Documentation : [сайт]. – 2026. – URL: <https://docs.docker.com/> (дата обращения: 29.01.2026).
- 6 Пряники : [сайт]. – 2026. – URL: <https://pryaniky.com/> (дата обращения: 24.01.2026).
- 7 IdeaScale : [сайт]. – Беркли, 2026. – URL: <https://ideascale.com/> (дата обращения: 24.01.2026).
- 8 Brightidea : [сайт]. – Сан-Франциско, 2026. – URL: <https://www.brightidea.com/> (дата обращения: 24.01.2026).
- 9 React Router : [сайт]. – 2026. – URL: <https://reactrouter.com/> (дата обращения: 29.01.2026).
- 10 Axios : [сайт]. – 2026. – URL: <https://axios-http.com/> (дата обращения: 29.01.2026).
- 11 React Hook Form : [сайт]. – 2026. – URL: <https://react-hook-form.com/> (дата обращения: 29.01.2026).
- 12 Yup : [сайт]. – 2026. – URL: <https://github.com/jquense/yup> (дата обращения: 29.01.2026).
- 13 HTTPS : [сайт]. – 2026. – URL: <https://developer.mozilla.org/ru/docs/Glossary/HTTPS> (дата обращения: 30.04.2026).
- 14 TypeORM : [сайт]. – 2026. – URL: <https://typeorm.io/> (дата обращения: 29.01.2026).
- 15 Class-validator : [сайт]. – 2026. – URL: <https://github.com/typestack/class-validator> (дата обращения: 29.01.2026).
- 16 Class-transformer : [сайт]. – 2026. – URL: <https://github.com/typestack/class-transformer> (дата обращения: 29.01.2026).
- 17 Bcrypt : [сайт]. – 2026. – URL: <https://www.npmjs.com/package/bcrypt> (дата обращения: 29.01.2026).
- 18 Nodemailer : [сайт]. – 2026. – URL: <https://nodemailer.com/> (дата обращения: 29.01.2026).

					<i>ДП 00.00.ПЗ</i>		
		ФИО	Подпись	Дата	<i>Список использованных источников</i>		
Разраб.		<i>Телица А.Д.</i>					
Пров.		<i>Сазонова Д.В.</i>					
Н. контр.		<i>Николайчук А.Н.</i>					
Утв.		<i>Блинова Е.А.</i>			<i>БГТУ 1-40 05 01, 2026</i>		
					<i>Лит. Лист Листов</i>		
					у	1	2

19 Passport JWT : [сайт]. – 2026. – URL: <https://www.passportjs.org/packages/passport-jwt/> (дата обращения: 29.01.2026).

20 PostgreSQL Documentation : [сайт]. – 2026. – URL: <https://www.postgresql.org/docs/> (дата обращения: 29.01.2026).

21 Software architecture : [сайт]. – 2026. – URL: https://en.wikipedia.org/wiki/Software_architecture (дата обращения: 25.01.2026).

22 Unified Modeling Language : [сайт]. – 2026. – URL: <https://www.uml.org/> (дата обращения: 25.01.2026).

23 Database model : [сайт]. – 2026. – URL: https://en.wikipedia.org/wiki/Database_model (дата обращения: 25.01.2026).

24 Draw.io : [сайт]. – 2026. – URL: <https://www.drawio.com/> (дата обращения: 25.01.2026).

25 Universally Unique Identifier : [сайт]. – 2026. – URL: https://en.wikipedia.org/wiki/Universally_unique_identifier (дата обращения: 25.01.2026).

26 Docker Compose Documentation : [сайт]. – 2026. – URL: <https://docs.docker.com/compose/> (дата обращения: 29.01.2026).

27 Data Transfer Object : [сайт]. – 2026. – URL: <https://martinfowler.com/eaaCatalog/dataTransferObject.html> (дата обращения: 25.01.2026).

28 Simple Mail Transfer Protocol : [сайт]. – 2026. – URL: https://en.wikipedia.org/wiki/Simple_Mail_Transfer_Protocol (дата обращения: 29.01.2026).

29 Endpoint : [сайт]. – 2026. – URL: <https://developer.mozilla.org/en-US/docs/Glossary/Endpoint> (дата обращения: 07.05.2026).

30 Stateless authentication : [сайт]. – 2026. – URL: <https://auth0.com/blog/stateless-auth-for-stateful-minds/> (дата обращения: 30.04.2026).

31 React Scripts : [сайт]. – 2026. – URL: <https://www.npmjs.com/package/react-scripts> (дата обращения: 29.01.2026).

32 Cross-env : [сайт]. – 2026. – URL: <https://www.npmjs.com/package/cross-env/v/7.0.3> (дата обращения: 29.01.2026).

33 DRY : [сайт]. – 2026. – URL: https://en.wikipedia.org/wiki/Don%27t_repeat_yourself (дата обращения: 07.05.2026).

ПРИЛОЖЕНИЕ А
Архитектурная схема приложения

ПРИЛОЖЕНИЕ Б
Диаграмма вариантов использования

ПРИЛОЖЕНИЕ В
Диаграмма развертывания

ПРИЛОЖЕНИЕ Г
Логическая схема базы данных

ПРИЛОЖЕНИЕ Д
Блок-схема алгоритма создания новой темы

ПРИЛОЖЕНИЕ Е
Экранная форма главной страницы

ПРИЛОЖЕНИЕ Ж
Таблица расчета затрат на разработку

ПРИЛОЖЕНИЕ И

Реализация сидера учетной записи администратора

```

import { Injectable } from '@nestjs/common';
import { InjectRepository } from '@nestjs/typeorm';
import { Repository } from 'typeorm';
import * as bcrypt from 'bcryptjs';
import { User } from '../entities/user.entity';
import { UserRole } from '../enums/user/user-role.enum';
import { UserStatus } from '../enums/user/user-status.enum';

@Injectable()
export class AdminSeed {
  constructor(
    @InjectRepository(User)
    private userRepository: Repository<User>,
  ) {}

  async seed() {
    const adminEmail = 'admin@ideaflow.com';

    const existingAdmin = await this.userRepository.findOne({
      where: { email: adminEmail }
    });

    if (!existingAdmin) {
      const hashedPassword = await bcrypt.hash('AdminPassword123!', 12);

      const adminUser = this.userRepository.create({
        email: adminEmail,
        password: hashedPassword,
        firstName: 'System',
        lastName: 'Administrator',
        role: UserRole.ADMIN,
        status: UserStatus.ACTIVE,
        isEmailVerified: true,
      });

      await this.userRepository.save(adminUser);
      console.log(' Admin user seeded');
    }
  }
}

```

ПРИЛОЖЕНИЕ К

Файл оркестрации контейнеров

```

services:
  # PostgreSQL database
  postgres:
    image: postgres:15-alpine
    container_name: ideaflow_db
    environment:
      POSTGRES_DB: ideaflow_db
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: 1234
    ports:
      - "5432:5432"
    volumes:
      - postgres_data:/var/lib/postgresql/data
    networks:
      - ideaflow_network
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U postgres"]
      interval: 5s
      timeout: 5s
      retries: 5

  # Backend API (NestJS)
  backend:
    build:
      context: ./idea-flow
      dockerfile: Dockerfile
    container_name: ideaflow_backend
    ports:
      - "3000:3000"
    environment:
      - NODE_ENV=development
      - DB_HOST=postgres
      - DB_PORT=5432
      - DB_USERNAME=postgres
      - DB_PASSWORD=1234
      - DB_NAME=ideaflow_db
      - JWT_SECRET=your-super-secret-jwt-key-for-development-123
      - FRONTEND_URL=http://localhost:3001
      - NODE_OPTIONS=--max-old-space-size=2048
    networks:
      - ideaflow_network
    depends_on:
      postgres:
        condition: service_healthy
    volumes:
      - ./idea-flow:/app
      - /app/node_modules
    command: npm run start:dev

```

```

    deploy:
      resources:
        limits:
          memory: 3G
        reservations:
          memory: 1G

# Frontend (React) - Development
frontend:
  build:
    context: ./idea-flow_web
    dockerfile: Dockerfile
    target: development
  container_name: ideafLOW_frontend
  ports:
    - "3001:3001"
  environment:
    - REACT_APP_API_URL=https://www.ideafLOW.of.by
    - CHOKIDAR_USEPOLLING=true
    - WDS_SOCKET_PORT=0
    - WDS_SOCKET_HOST=localhost
    - NODE_OPTIONS=--max-old-space-size=4096
    - GENERATE_SOURCEMAP=false
    - WATCHPACK_POLLING=true
    - FAST_REFRESH=false
    - CI=false
    - DISABLE_ESLINT_PLUGIN=true
    - WATCHPACK_WATCHER_LIMIT=20
  networks:
    - ideafLOW_network
  volumes:
    - ./idea-flow_web:/app
    - /app/node_modules
  stdin_open: true
  tty: true
  depends_on:
    - backend
  deploy:
    resources:
      limits:
        memory: 4G
      reservations:
        memory: 2G

volumes:
  postgres_data:

networks:
  ideafLOW_network:
    driver: bridge

```